MULTIMODAL LEARNING

多模态学习

课程讲义

多模态学习基础
深度学习的数学基础
计算机视觉

2026 年秋季

目录

第 1 章　多模态学习基础 **1**
1.1　任务与概率模型 1
1.2　多模态学习的主要方法 2
1.3　不同模态提供的信息 5

第 2 章　深度学习的数学基础 **6**
2.1　神经网络与反向传播 6
2.2　激活函数与门控 8
2.3　归一化 11
2.4　损失函数与优化 15
2.5　正则化与训练 17

第 3 章　计算机视觉 **21**
3.1　卷积 21
3.2　池化、感受野与分类 23
3.3　CNN 的计算量与结构 25
3.4　ResNet 与 SENet 27
3.5　轻量化 CNN 28
3.6　大核卷积与动态卷积 30
3.7　数据增强 32
3.8　目标检测与评价指标 34
3.9　R-CNN 系列 35
3.10　YOLO 系列 38
3.11　检测损失函数 40
3.12　语义分割与 FCN 42
3.13　上采样与 U-Net 43

参考文献 **46**

符号说明

记号	含义
$x^{(m)}$，y	第 m 种模态的观测；预测目标。上标 (m) 不是幂。
N，B	数据集样本数；一个 mini-batch 的样本数。
d，h，T	特征维度；中间层宽度；序列长度。
C，H，W	图像张量的通道数、高、宽；分类类别数另写 K。
θ，η，λ	模型参数；学习率；正则强度。
$\odot$，$[u;v]$	逐元素乘法；沿特征维拼接。
$\sigma(x)$	默认表示 sigmoid；标准差写为 $\sqrt{v}$，避免混用。

单样本推导默认使用**列向量**，线性层写作 $Wx+b$。批量代码把样本放在前面的维度，张量通常为 $[B,d]$ 或 $[B,T,d]$。PyTorch 的 `Linear(d,h)` 存储 $[h,d]$ 的权重，计算 $XW^\top+b$；两种写法描述同一运算。

第 1 章　多模态学习基础

1.1　任务与概率模型

模态（modality）是信息的类型，例如图像、文本和声音。不同模态往往描述同一个对象或事件。一段视频记录了人物的动作，声音提供了说话内容，字幕又把语言写成文字。多模态学习研究怎样把这些信息结合起来，用于识别、问答或生成。

设数据为

$$\mathcal{D} = \left\{ \left(x_i^{(1)}, \ldots, x_i^{(M)}, y_i \right) \right\}_{i=1}^{N}. \tag{1.1}$$

其中 $x_i^{(m)}$ 表示第 i 个样本的第 m 种模态，y_i 是预测目标。一幅图像和它的描述可以组成一个样本，二者分别保存为像素数组和词序列，各自有自己的长度和维度。数据中的配对关系说明哪些观测描述了同一件事。训练和实际使用时可用的模态，也要在任务中分别说明。

任务	输入 $\to$ 输出	统一数学描述
图像／视频描述	视觉 $\to$ 文本	$p_\theta(w_{1:T} \mid v)$
视觉问答／对话	视觉、问题 $\to$ 答案	$p_\theta(a \mid v, q, h)$，h 为历史
指代表达定位	图像、表达 $\to$ 区域	$p_\theta(b \mid v, q)$，b 为框或掩码
视频时刻定位	视频、句子 $\to$ 时间段	$p_\theta(t_s, t_e \mid v, q)$
文本生成图像	文本 $\to$ 图像	$p_\theta(v \mid w)$
视觉语言导航	观测、指令 $\to$ 动作	$\pi_\theta(a_t \mid o_{\le t}, q)$

表 1.1　常见多模态任务的输入、输出及其概率表示。

例如，图像描述可按链式法则分解为

$$p_\theta(w_{1:T} \mid v) = \prod_{t=1}^{T} p_\theta(w_t \mid w_{<t}, v). \tag{1.2}$$

生成句子时，模型先根据图像预测第一个 token，再结合图像和已经生成的内容预测后续 token。式中的每一项对应其中一步。定位任务则要输出具体区域或时间段，因此编码后的特征还需保留位置或时间信息。

> **数学直觉　|　任务的输出**
> 图像分类输出类别，图像检索输出匹配分数，目标定位输出坐标，图像描述输出词序列。这些任务可以使用相似的图像编码器，再接上各自的预测层。损失函数也随输出形式选择，例如类别使用交叉熵，坐标可以使用回归损失。

编码器与预测模型

对两种模态，最基本的计算过程可写为

$$z^{(v)} = f_v(x^{(v)}), \qquad z^{(t)} = f_t(x^{(t)}), \qquad \hat{y} = g_\theta\Big(\mathcal{F}(z^{(v)}, z^{(t)})\Big). \tag{1.3}$$

图像编码器 f_v 和文本编码器 f_t 分别把输入转换为特征。接着，$\mathcal{F}$ 结合两组特征，g_θ 根据结合后的结果作出预测。特征 z 可以是一个向量，也可以是一组保留位置的向量：图像中每个 patch 对应一个向量，文本中每个 token 对应一个向量。

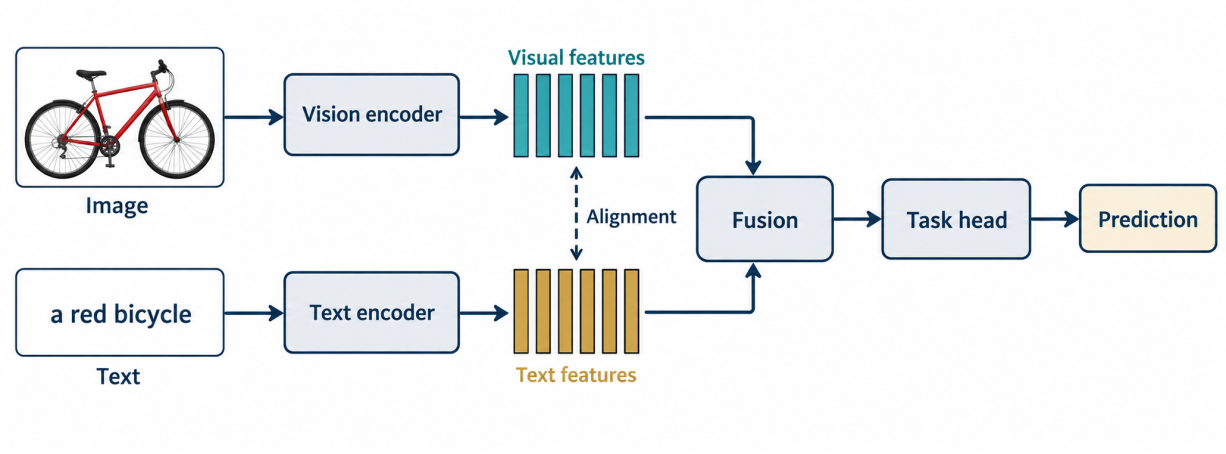


图 1.1 双模态预测模型。实线表示计算顺序，虚线表示两组特征之间的对应关系。

图像与文本通常使用不同的编码器。图像编码器处理像素的空间关系，例如边缘、纹理和遮挡；文本编码器处理词语的顺序、含义以及上下文。两组特征之间的对应关系，则通过配对数据和后续训练学习。

两种模态描述同一事件时，记录的粒度也常常不同。一段视频可能有数百帧，而描述只有十几个词。“走进房间”对应一段连续动作，“红色自行车”对应图像中的一个区域。对齐时需要寻找这样的局部对应。

有些模型在训练时同时使用图像和文本，在部署时只接收图像。这时，文本的作用是帮助图像编码器学习。模型的输入设计和训练目标需要与这种使用方式配合。

> **备注 | 单模态基线**
> 单模态模型可以作为比较的起点。例如，先测量问答模型只读问题时的成绩，再加入图像，观察哪些问题得到了改善。还可以遮蔽或打乱图像，检查预测是否随之改变。解释这类实验时，要同时考虑信息被移除和输入分布改变这两个影响。

1.2 多模态学习的主要方法

多模态学习常按表征、对齐、转译、融合和协同学习五个方面讨论。[1] 它们分别关注特征的表示方式、局部对应、跨模态生成、信息组合和训练中的知识迁移。一套模型通常同时涉及其中几项，例如图像问答既需要理解图文特征，也需要找出问题所指的区域。

表征学习

联合表征把多个模态合成一个表示：

$$z = f_\theta\Big(x^{(v)}, x^{(t)}\Big). \tag{1.4}$$

它适合需要联合推理的任务。**协调表征**分别编码，再通过相似度或约束建立关系：

$$z^{(v)} = f_v(x^{(v)}), \quad z^{(t)} = f_t(x^{(t)}), \quad s(v,t) = \frac{{z^{(v)}}^\top z^{(t)}}{\left\|z^{(v)}\right\|_2 \left\|z^{(t)}\right\|_2}. \tag{1.5}$$

协调表征常用于图文检索。系统可以提前编码所有候选图像，收到一句文本后，再计算它与各图像的相似度。匹配关系由向量整体表达，训练主要约束这些向量之间的距离或夹角。

以批内对比学习为例，给定 B 个配对样本，s_{ij} 表示第 i 张图与第 j 段文本的相似度，可定义

$$\mathcal{L}_{v\to t} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp(s_{ii}/\tau)}{\sum_{j=1}^{B}\exp(s_{ij}/\tau)}, \qquad \tau > 0. \tag{1.6}$$

它把“找配对文本”变成一个 B 类分类问题。对称方向 $\mathcal{L}_{t\to v}$ 交换行列，两者取平均即可得到一个双向目标。

数学直觉 | 对比学习

式 (1.6) 同时比较正确文本和其他候选文本。分子中的正确配对分数越高，损失越小；分母中的其他分数越高，损失越大。这样，模型既学习配对关系，也学习区分不同样本。较小的温度 τ 会让 softmax 更集中，使分数较高的负样本获得更多关注。

备注

一张图像可能有多种合理描述。例如，同一批次中的两张自行车照片，都可能符合“路边的一辆自行车”。构造对比学习数据时，可以去除重复样本，或把多个合理配对都标为正例。

模态对齐

设视觉特征为 $U \in \mathbb{R}^{T_v\times d}$，语言特征为 $V \in \mathbb{R}^{T_t\times d}$。对齐关注一个关系矩阵 $A \in \mathbb{R}^{T_t\times T_v}$，其中 A_{ij} 描述文本元素 i 与视觉元素 j 的关联。

显式对齐直接预测或计算对应关系，例如时间同步、词语与区域的匹配。**隐式对齐**把对应关系作为完成其他任务的中间变量，例如为了生成答案而学习关注哪些图像区域。

一个常见的软对齐写法是交叉注意力：

$$Q = VW_Q, \quad K = UW_K, \quad Z = UW_V, \qquad A = \mathrm{softmax}_{\mathrm{row}}\left(\frac{QK^\top}{\sqrt{d_k}}\right), \quad O = AZ. \tag{1.7}$$

$Q \in \mathbb{R}^{T_t\times d_k}$，$K \in \mathbb{R}^{T_v\times d_k}$，因此每个词都得到一个对视觉元素的权重分布。$Z$ 是值向量，符号 W_V 中的 V 表示 value；它与上文语言特征矩阵 V 的角色不同。

对存在顺序约束的两段序列，还可以用动态规划计算单调对齐。令 $c(i,j)$ 为局部匹配代价，动态时间规整（DTW）的核心递推为

$$D(i,j) = c(i,j) + \min\{D(i-1,j), D(i,j-1), D(i-1,j-1)\}. \tag{1.8}$$

初始化 $D(0,0)=0$，其余第零行与第零列设为 $+\infty$。每一步从三个允许的前驱位置中选取代价最小者，再加上当前匹配代价。计算完成后沿最优前驱回溯，就得到两段序列的对齐路径。

备注 | 注意力对齐

注意力矩阵的每一行给出一个权重分布，一个词可以同时参考多个图像区域。对于有时间先后的动作，例如“先进入房间再坐下”，可以另外加入单调路径或时间监督，使对齐结果符合事件顺序。

跨模态生成

跨模态生成学习条件分布 $p_\theta(x^{(b)} \mid x^{(a)})$。输入一段文字时，模型可以生成多幅符合描述的图像；输入同一视频时，也可以生成详略不同的描述。条件分布用来表示这些可能的输出。

数学直觉 | 条件均值

用平方误差预测连续输出时，总体最优解为条件均值 $f^*(x) = \mathbb{E}[Y \mid X = x]$。如果同一句描述对应两个差异很大的姿态，平均后的图像可能变得模糊。要生成清晰且多样的结果，模型需要学习这些可能结果的分布，并从中生成一个具体样本。

多模态融合

融合关心**何时交互、以什么形式交互**。设 $z_v \in \mathbb{R}^{d_v}$、$z_t \in \mathbb{R}^{d_t}$。

方式	典型写法	特点
特征级融合	$g([z_v; z_t])$	容易实现；交互能力取决于后续 g
交互式融合	$g(\mathcal{F}(z_v, z_t))$	用注意力、门控等显式建模关联
决策级融合	$\sum_m \alpha_m p_m(y \mid x^{(m)})$	模态可独立预测；要求 $\alpha_m \geq 0$ 且和为 1

表 1.2 “早、中、晚融合”描述相对位置，实际模型可能混合多种方式。

若在拼接后只接一个线性层，输出是 $W_v z_v + W_t z_t + b$，两个模态的贡献是相加的。若需要表达“这个词与这个区域同时出现才重要”，可以引入乘性交互，例如双线性评分

$$s(z_v, z_t) = z_v^\top A z_t, \qquad A \in \mathbb{R}^{d_v \times d_t}. \tag{1.9}$$

矩阵 A 显式编码两种特征之间的成对关系；参数量较大时，可以令 $A = PQ^\top$ 做低秩分解。第二章的门控机制也会再次出现“乘法产生交互”的思想。

协同学习

协同学习利用一种模态的信息帮助另一种模态训练。例如，模型可以在训练时参考视频中的声音，学习更好的视觉特征，随后仅凭图像完成预测。不同模态之间可以通过逐样本配对、共享标签或共同概念建立联系。

例如，训练时用图像和文本构建教师分布 $p_T(y \mid v, t)$，而学生只接收图像。一个简化的蒸馏目标为

$$\mathcal{L}_{\text{student}} = \mathcal{L}_{\text{sup}}(p_S, y) + \lambda D_{\text{KL}}(p_T(\cdot \mid v, t) \,\|\, p_S(\cdot \mid v)) \,. \tag{1.10}$$

计算这一损失时，教师参数固定。KL 项使学生的类别分布接近教师，监督损失则使它符合真实标签。学生最终只接收图像，因此它能学到的知识取决于图像中可辨认的内容。

数学直觉 | 融合与协同学习

融合描述模型如何一起使用多种输入，协同学习描述一种模态如何帮助另一种模态训练。例如，图文教师使用融合后的特征作出判断，再通过蒸馏训练只接收图像的学生。这个过程同时包含融合与协同学习。

1.3 不同模态提供的信息

一种新模态能否帮助预测，取决于它提供了什么信息。设图像为 V，文本为 T，分类目标为 Y。在已经观察图像后，文本带来的额外信息可以写为

$$I(Y;T \mid V) = H(Y \mid V) - H(Y \mid V,T) \geq 0. \tag{1.11}$$

如果文本消除了图像中的歧义，观察二者后，标签的不确定性就会下降。如果文本只是重复图像中已有的内容，额外信息就较少。重复信息还可能在某种输入受噪声干扰时帮助判断。

上式是在真实联合分布下讨论的信息量。模型训练还受到数据量和优化过程的影响：新增输入会增加需要学习的关系，错误配对也会带来错误监督。因此，实验中需要比较加入新模态前后的表现，并检查模型实际使用了哪些信息。

异或问题

令 V, T 为相互独立的公平二元变量，目标 $Y = V \oplus T$。给定 $V = 0$ 后，Y 仍取决于尚未观察到的 T；给定 $V = 1$ 也是如此。只看 T 时情况相同。同时观察二者，就可以计算异或值并确定答案。这个例子说明，任务有时需要利用两个输入之间的关系。

数学直觉 | 共享信息与模态特有信息

图像和文本可以共享物体类别这样的信息，同时保留各自的细节。图像中的颜色和位置、文本中的动作顺序，都可能影响最终答案。模型可以使用共享特征表达共同内容，再用各模态自己的特征保存这些细节。

模型分析

1. 信息：每个模态提供什么独有线索？标签能否被单模态捷径预测？
2. 结构：对应发生在样本、区域、词语还是时间片段层面？
3. 目标：训练损失鼓励相似、定位、生成，还是最终任务正确？
4. 部署：模态缺失、噪声增强或分布改变时，模型依赖什么？

第 2 章　深度学习的数学基础

2.1　神经网络与反向传播

线性层与非线性激活

令 $h^{(0)} = x$，一个多层感知机（MLP）可以写成

$$a^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}, \qquad h^{(\ell)} = \phi\left(a^{(\ell)}\right). \tag{2.1}$$

若 $h^{(\ell-1)} \in \mathbb{R}^{d_{\ell-1}}$、$h^{(\ell)} \in \mathbb{R}^{d_\ell}$，则 $W^{(\ell)} \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$，偏置 $b^{(\ell)} \in \mathbb{R}^{d_\ell}$。输出层是否使用激活，取决于后续的概率模型与损失。

若所有 ϕ 都是恒等映射，两层计算变为

$$h^{(2)} = W^{(2)}(W^{(1)}x + b^{(1)}) + b^{(2)} \tag{2.2}$$

$$= \underbrace{W^{(2)}W^{(1)}}_{W_{\text{eff}}} x + \underbrace{W^{(2)}b^{(1)} + b^{(2)}}_{b_{\text{eff}}}. \tag{2.3}$$

将两层权重相乘、偏置合并后，就得到一个新的仿射层。这个过程可以逐层重复，因此纯仿射网络可以合并成一层。加入非线性激活后，网络才能随着输入变化形成更丰富的函数。

> **数学直觉　|　ReLU 的分段线性形式**
> 对固定输入，把 ReLU 的开关写成对角矩阵 $D(x)$，对角元为 0 或 1，则
>
> $$W_2\,\mathrm{ReLU}(W_1x + b_1) + b_2 = W_2D(x)(W_1x + b_1) + b_2.$$
>
> 在开关模式不变的区域，网络是仿射函数；跨过边界后，$D(x)$ 改变，局部斜率也随之改变。深度网络通过复合这些分区，表达更复杂的函数。

权重、偏置和归一化层的缩放系数是可学习参数，由训练中的梯度更新。层数、宽度和学习率等设置称为超参数，通常通过验证集选择。前者决定一个已选网络如何计算，后者决定使用多大的网络以及如何训练它。

通用逼近定理

对紧致集合 $\mathcal{K} \subset \mathbb{R}^d$ 上的连续函数 f，使用合适的非线性激活（如 sigmoid 或 ReLU）与足够宽的单隐藏层网络，可以使

$$\sup_{x \in \mathcal{K}} |f(x) - f_\theta(x)| < \varepsilon, \qquad \varepsilon > 0. \tag{2.4}$$

这个定理说明：给定连续函数 f 和误差 ε，可以找到足够大的网络及一组参数，使逼近误差小于 ε。所需的宽度随函数和误差要求变化。定理讨论这些参数的存在性，实际训练还需要通过优化找到合适的参数。

一维连续分段线性函数可以用 ReLU 的“折点”表示：

$$p(x) = a + bx + \sum_{k=1}^{K} c_k \,\mathrm{ReLU}(x - t_k). \tag{2.5}$$

t_k 决定折点位置，c_k 决定经过该折点后斜率改变多少。因为 $\frac{d}{dx}\,\mathrm{ReLU}(x - t_k) = \mathbf{1}[x > t_k]$（折点处另选次梯度），每加入一个单元，就能在指定位置修改斜率。

例如，在区间 $[0, 3]$ 上，对 x^2 在整数节点做线性插值，有

$$p(x) = x + 2\,\mathrm{ReLU}(x - 1) + 2\,\mathrm{ReLU}(x - 2). \tag{2.6}$$

三段斜率依次为 1、3、5，且 $p(0), p(1), p(2), p(3) = 0, 1, 4, 9$。每个 ReLU 单元从自己的折点开始增加斜率，于是几段直线连成了对 x^2 的近似。增加插值节点，就可以让折线更贴近原曲线。式 (2.5) 展示了这一维构造。

> **数学直觉 | 网络深度**
> 一个很宽的浅层网络可以表达复杂函数。深层网络则把计算分成多步，例如先识别边缘，再组合成部件，最后识别物体。当目标函数也有这样的组合过程时，多层网络可能用较少的参数完成表示。

> **备注 | 表达能力、优化与泛化**
> 表达能力描述网络可以表示哪些函数，优化研究如何从训练数据找到参数，泛化考察模型在新样本上的表现。通用逼近定理主要讨论表达能力。网络的训练误差和验证误差，则分别帮助我们观察优化与泛化。

反向传播

定义 $\delta^{(\ell)} = \partial \mathcal{L} / \partial a^{(\ell)}$。链式法则给出

$$\delta^{(\ell)} = \left((W^{(\ell+1)})^{\top} \delta^{(\ell+1)} \right) \odot \phi'(a^{(\ell)}), \tag{2.7}$$

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \delta^{(\ell)} (h^{(\ell-1)})^{\top}, \qquad \frac{\partial \mathcal{L}}{\partial b^{(\ell)}} = \delta^{(\ell)}. \tag{2.8}$$

第二式是误差信号与输入激活的外积。对某个权重而言，梯度等于它接收的输入值，乘以对应输出单元的误差信号。批量训练会对各样本的梯度求和或平均，具体形式与损失函数的计算方式一致。

深层网络的输入输出 Jacobian 含有多个矩阵的连乘：

$$\frac{\partial h^{(L)}}{\partial x} = D^{(L)} W^{(L)} \cdots D^{(1)} W^{(1)}, \quad D^{(\ell)} = \mathrm{diag}\big(\phi'(a^{(\ell)})\big). \tag{2.9}$$

连乘中某些方向持续缩小，会造成梯度消失；持续放大则可能导致梯度爆炸。激活函数、初始化、归一化与残差连接都在影响这条传播路径。

对残差块 $h_{\ell+1} = h_\ell + F_\ell(h_\ell)$，有

$$\frac{\partial h_{\ell+1}}{\partial h_\ell} = I + J_{F_\ell}. \tag{2.10}$$

恒等项让梯度可以直接通过这个块。在初始化时，如果残差分支 F_ℓ 及其 Jacobian 都较小，块的输出接近输入，梯度也容易保持原有尺度。随着训练进行，残差分支逐步学习需要添加的变化。

算法实现 | 前向计算与反向传播

```
import torch
from torch import nn
import torch.nn.functional as F

net = nn.Sequential(nn.Linear(d, h), nn.GELU(),
                    nn.Linear(h, num_classes))
logits = net(x)                      # x: [B, d]
loss = F.cross_entropy(logits, y)    # y: [B], int64
loss.backward()
grad_norm = net[0].weight.grad.norm()
```

这里假设 `x,y,d,h,num_classes` 已给定。先确认输出形状、损失和梯度有限，再接入完整训练循环。

2.2 激活函数与门控

Sigmoid、Tanh 与 ReLU

函数	定义与导数要点	对优化的影响
Sigmoid	$\sigma(x) = 1/(1+e^{-x})$	导数 $\sigma(x)(1-\sigma(x)) \le 1/4$；两端饱和
Tanh	$\tanh x$	导数 $1-\tanh^2 x$；零中心，但仍会饱和
ReLU	$\max(0, x)$	正半轴导数为 1；负半轴为 0
Leaky ReLU	$\max(\alpha x, x)$，$0 < \alpha < 1$	负半轴保留斜率；PReLU 将斜率设为可学习参数

表 2.1 激活的选择改变表达形式，也改变误差传播路径。

饱和指较大输入变化只造成很小输出变化，因此局部导数接近零。Sigmoid 可作为 Bernoulli 输出概率的参数化，但作为深层隐藏激活时，多层小导数相乘容易削弱梯度。

ReLU 在负半轴上的梯度为零。如果某个单元对训练样本长期输出零，它就很难通过这条通路更新。Leaky ReLU 给负半轴保留一个较小的斜率，使这些输入仍能产生梯度。

非零中心输入会怎样影响单个神经元的更新？令 $a = w^\top x + b$，并记 $\delta = \partial \mathcal{L}/\partial a$，则

$$\frac{\partial \mathcal{L}}{\partial w_j} = \delta x_j. \tag{2.11}$$

若对一个样本所有 $x_j > 0$，各权重梯度的符号便由同一个 δ 决定。某些最优下降方向需要不同坐标向相反方向调整，这种相关性会使单样本更新更曲折。

备注 | 批量梯度

对一个 mini-batch，梯度为 $\sum_i \delta_i x_{ij}$。不同样本的误差信号 δ_i 可以有正有负，相加后各坐标的梯度也可以取不同符号。批量更新的方向因此由整个批次共同决定。

GELU 与 SiLU

GELU [2] 用标准正态分布的累积分布函数 Φ 对输入加权：

$$\mathrm{GELU}(x) = x\Phi(x), \tag{2.12}$$

$$\mathrm{GELU}'(x) = \Phi(x) + x\varphi(x), \tag{2.13}$$

其中 φ 是标准正态密度。较大的正输入基本保持原值，较大的负输入被压到零附近，零附近则平滑过渡。GELU 在一段负输入区间内会先下降再上升。

> **数学直觉 | GELU 的概率解释**
>
> 设 $m \mid x \sim \mathrm{Bernoulli}(\Phi(x))$，则
>
> $$\mathbb{E}[mx \mid x] = x\Phi(x).$$
>
> 这个式子把 GELU 理解为随机门控的平均结果：输入越大，通过门的概率越高。实际前向计算直接使用这个期望值，因此给定输入就得到确定的输出。

常用近似为

$$\mathrm{GELU}(x) \approx \frac{x}{2}\left[1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right)\right]. \tag{2.14}$$

另一个常见门控族是 $\mathrm{Swish}_\beta(x) = x\sigma(\beta x)$。取 $\beta = 1$ 得到 SiLU：

$$\mathrm{SiLU}(x) = x\sigma(x), \qquad \mathrm{SiLU}'(x) = \sigma(x) + x\sigma(x)(1 - \sigma(x)). \tag{2.15}$$

GELU 与 SiLU 都把输入乘以一个平滑权重。GELU 使用正态分布函数 $\Phi(x)$，SiLU 使用 sigmoid $\sigma(x)$。另一个常见的 GELU 近似是 $x\sigma(1.702x)$，其中 sigmoid 的输入多了一个缩放系数。

> **算法实现 | GELU 与 SiLU 的实现**
>
> ```
> gelu_exact = F.gelu(x, approximate="none")
> gelu_approx = F.gelu(x, approximate="tanh")
> silu = F.silu(x)
> ```
>
> 上面分别调用 GELU 的精确形式、双曲正切近似和 SiLU。复现模型时使用其原有的激活设置；试验新的激活函数后，再比较训练和验证结果。

SwiGLU

普通前馈块通常写作 $W_o\phi(W_i x + b_i) + b_o$。SwiGLU [3] 使用两次输入投影，并逐元素相乘。省略偏置时：

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(x) = W_o\big(\mathrm{SiLU}(W_g x) \odot (W_v x)\big), \tag{2.16}$$

其中 $W_g, W_v \in \mathbb{R}^{h\times d}$，$W_o \in \mathbb{R}^{d\times h}$。两条输入分支都把 x 映射为 h 维向量：$W_v x$ 提供待处理的值，$\mathrm{SiLU}(W_g x)$ 给出各分量的乘法系数。系数可以小于零或大于 1，因此既能改变幅度，也能改变符号。

> **数学直觉 | 门控中的乘法**
>
> 暂时去掉 SiLU 后，第 k 个分量成为 $(w_{g,k}^\top x)(w_{v,k}^\top x)$。展开后会出现 $x_i x_j$ 这样的项，使不同输入分量共同影响输出。加入 SiLU 后，门控系数又随输入平滑变化。

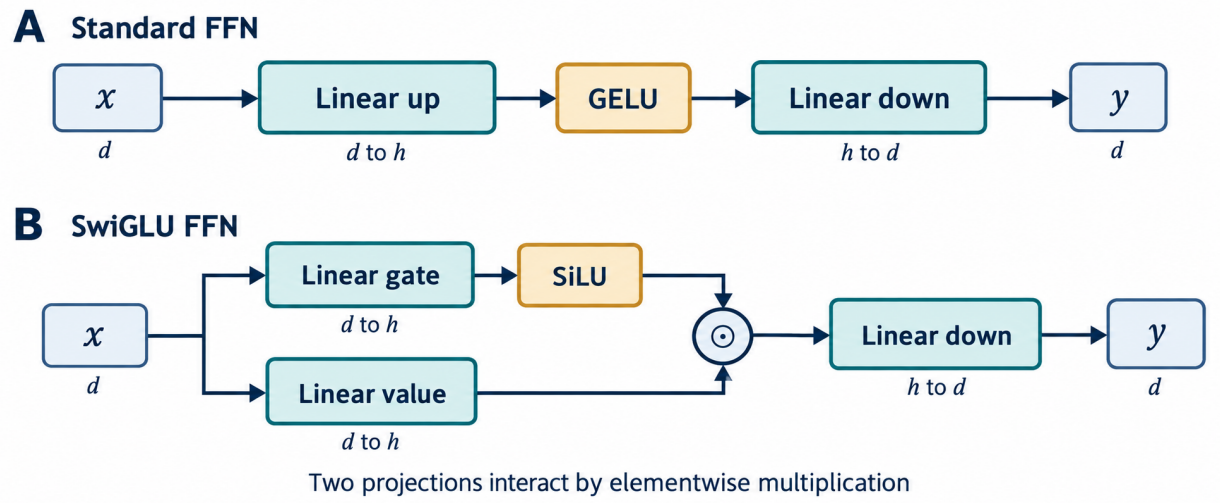


图 2.1 普通 FFN 与 SwiGLU FFN。d 为输入输出维度，h 为中间宽度；下图仅门控分支经过 SiLU，随后与值分支逐元素相乘。

SwiGLU 的参数量

忽略偏置，普通 FFN 有约 $2dh_{\text{plain}}$ 个参数，SwiGLU FFN 有约 $3dh_{\text{gated}}$ 个参数。若比较时保持参数规模近似相同，应取

$$h_{\text{gated}} \approx \frac{2}{3} h_{\text{plain}}. \tag{2.17}$$

例如，普通 FFN 使用 $h_{\text{plain}} = 4d$ 时，SwiGLU 可以取 $h_{\text{gated}} \approx 8d/3$，再根据实现要求取整。这样，两者的参数量大致相当；若保持相同中间宽度，SwiGLU 的参数量约为普通 FFN 的 1.5 倍。

算法实现 | SwiGLU 的实现

```
class SwiGLU(nn.Module):
    def __init__(self, d, h):
        super().__init__()
        self.gate = nn.Linear(d, h, bias=False)
        self.value = nn.Linear(d, h, bias=False)
        self.out = nn.Linear(h, d, bias=False)

    def forward(self, x):              # [..., d]
        g = F.silu(self.gate(x))       # [..., h]
        v = self.value(x)              # [..., h]
        return self.out(g * v)         # [..., d]
```

线性层作用在最后一维，因此输入可以是 $[B, d]$ 或 $[B, T, d]$。对于序列输入，每个 token 都经过同一套 FFN，保留原来的位置。

激活、门控与 FFN

- 标量激活：ReLU、GELU、SiLU，逐元素改变数值。
- 门控单元：用两个投影分支的乘积实现输入相关的调节。
- FFN 模块：包含投影、激活／门控与输出投影的完整子网络。

把使用 GELU 的 FFN 改成 SwiGLU 时，需要增加一条输入投影分支，并重新设置中间宽度。上面的代码展示了这几个部分怎样连接。

> **备注 | 与多模态的联系**
> 图像 token 和文本 token 都可以使用逐位置 FFN 来处理自身特征。图像区域与词语之间的信息交换，则由前后的注意力层或融合层完成。

2.3 归一化

训练中，各层特征的数值尺度会影响后续计算和梯度。归一化层先从一组元素中计算均值、方差等统计量，再按这些统计量调整输入。不同归一化方法的主要区别，在于选取哪一组元素进行计算。

对统计集合 S，统一写为

$$\mu_S = \frac{1}{|S|}\sum_{i\in S} x_i, \quad v_S = \frac{1}{|S|}\sum_{i\in S}(x_i-\mu_S)^2, \quad y_i = \gamma_i \frac{x_i-\mu_S}{\sqrt{v_S+\varepsilon}} + \beta_i. \tag{2.18}$$

先减去均值，再除以带稳定项的标准差，就完成了标准化。ε 用来处理方差很小的情况。之后，模型通过可学习的 γ, β 调整输出的尺度和偏移，使结果适合下一层。

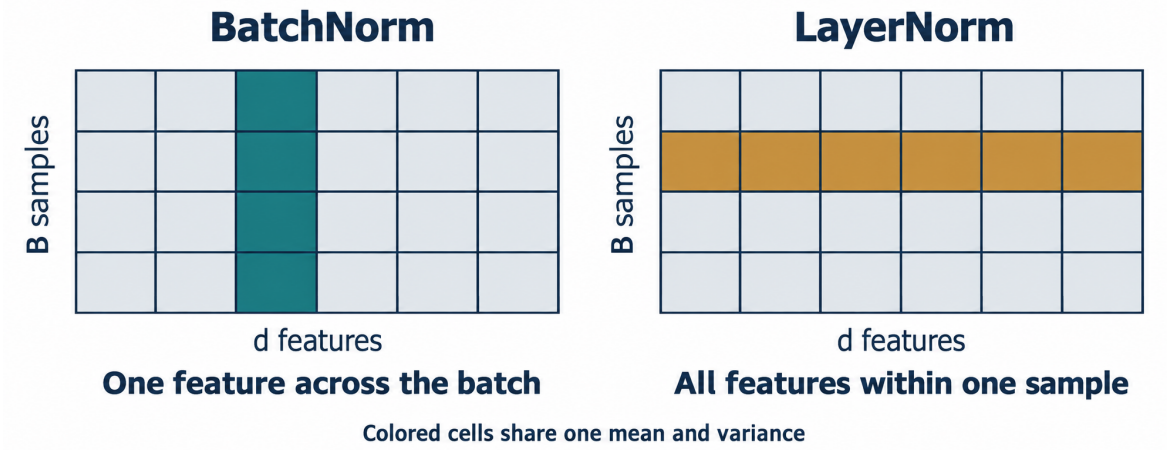


图 2.2 对 $X \in \mathbb{R}^{B\times d}$，BN 按列收集统计量，LN 按行收集统计量。着色区域中的元素共享一组均值与方差。

BatchNorm 与 LayerNorm

BatchNorm [4]： 对 $X \in \mathbb{R}^{B\times d}$，每个特征使用批次中的 B 个值，μ, v 的形状为 $[1, d]$。同一样本的输出会受到批内其他样本影响。

LayerNorm [5]： 对同一矩阵，每个样本在自己的 d 个特征上归一化，μ, v 的形状为 $[B, 1]$。对序列张量 $[B, T, d]$，常用做法是每个 token 独立沿 d 归一化。

归一化的计算范围

对 GN，$G = 1$ 时，一个样本的全部通道和空间位置组成同一组；$G = C$ 时，每个通道各自成组，统计范围与 IN 相同。比较具体实现时，还要看缩放与偏移参数作用在哪些位置，以及 ε 的取值。

方法	输入布局	求统计量的轴	统计量与仿射参数
BN	$[B, C, H, W]$	B, H, W	每个通道一组统计量；γ, β 各 C 个
LN（全特征）	$[B, C, H, W]$	C, H, W	每个样本一组；参数可覆盖整个归一化形状
IN	$[B, C, H, W]$	H, W	每个样本、每个通道一组；仿射参数可选
GN	$[B, G, C/G, H, W]$	$C/G, H, W$	每个样本、每组一套统计量；通常按通道仿射
LN（token）	$[B, T, d]$	d	每个 token 一组统计量；通常有 d 个缩放与偏移
RMSNorm	$[B, T, d]$	d 的平方均值	不减均值；通常有 d 个缩放参数

表 2.2 GN 要求通道数可被组数 G 整除。不同实现的仿射参数布局应单独确认。

训练与推理

BN 在训练时使用当前 batch 的统计量，并更新运行均值与方差；推理时通常使用训练中积累的统计量。batch 很小时，当前统计量的波动较大。部署数据与训练数据相差较大时，积累的统计量也可能不再适合新输入。

LN、GN 和标准的逐样本 IN 在单个样本内部计算。LN 在训练和推理时都使用同一套归一化步骤。

算法实现 | 归一化层的实现

```
# image: [B, C, H, W]
bn = nn.BatchNorm2d(C)            # reduce B,H,W
gn = nn.GroupNorm(G, C)           # reduce C/G,H,W

# tokens: [B, T, d]
ln = nn.LayerNorm(d)              # reduce last dim
tokens_out = ln(tokens)
```

`LayerNorm(d)` 沿最后一维计算。对 $[B, T, d]$ 的输入，这正好是每个 token 的特征维。如果图像采用 $[B, C, H, W]$ 布局，而希望沿 C 归一化，可以先把通道维移到最后，再调用 `LayerNorm(C)`。

备注 | 实现细节

式 (2.18) 的方差除以 $|S|$，PyTorch 的 BN 在训练前向计算中采用这一形式。更新运行方差时，BN 使用无偏估计。IN 是否保存并使用运行统计量由配置决定。

RMSNorm

对 $x \in \mathbb{R}^d$，RMSNorm [6] 的常见形式为

$$\mathrm{RMSNorm}(x) = \gamma \odot \frac{x}{\sqrt{\frac{1}{d}\sum_{j=1}^{d} x_j^2 + \varepsilon}}. \tag{2.19}$$

RMSNorm 直接用输入的均方根缩放各分量，通常只学习缩放参数 γ。LN 则先减去均值，再计算标准差。这两个分母可以通过下面的等式联系起来：

$$\frac{1}{d}\sum_j x_j^2 = \underbrace{\frac{1}{d}\sum_j (x_j-\mu)^2}_{v} + \mu^2 \tag{2.20}$$

等式右侧由方差 v 和均值的平方 μ^2 组成。因此，输入的整体偏移会影响 RMSNorm 的分母；LN 在减去均值后保留各分量相对均值的差异。

令 $P = I - \frac{1}{d}\mathbf{1}\mathbf{1}^\top$。忽略 ε 和仿射参数，在分母非零时，

$$\mathrm{LN}_0(x) = \sqrt{d}\,\frac{Px}{\|Px\|_2}, \qquad \mathrm{RMSNorm}_0(x) = \sqrt{d}\,\frac{x}{\|x\|_2}. \tag{2.21}$$

Px 表示减去均值后的向量，它的各分量之和为零。LN 将这个向量的长度调整为 $\sqrt{d}$；RMSNorm 则直接调整原向量的长度。两种计算都使用整个向量的信息来确定缩放系数。

数学直觉 | 缩放与平移

忽略稳定项并假设分母非零时，对 $a > 0$，两种方法都满足整体缩放后的输出保持不变。输入乘以 a，分母也会乘以 a，二者正好抵消。

LN 还满足 $\mathrm{LN}_0(x + b\mathbf{1}) = \mathrm{LN}_0(x)$，因为所有分量共同增加的 b 会在减均值时被消去。例如 $(1, 2, 3)$ 与 $(101, 102, 103)$ 的 LN 输出相同，RMSNorm 输出则随原向量方向的变化而改变。实际计算保留 ε 时，缩放不变性是近似成立的。

算法实现 | RMSNorm 的实现

```
def rms_norm(x, weight, eps=1e-6):
    xf = x.float()  # FP32 accumulation for fp16/bf16
    inv_rms = torch.rsqrt(xf.square().mean(-1, keepdim=True)
                          + eps)
    return (xf * inv_rms * weight.float()).to(x.dtype)
```

`weight` 的形状为 $[d]$。此代码展示常见低精度输入的计算主干，实际项目可使用框架中的标准或融合实现。

AdaLN

普通 LN 使用与样本无关的可学习参数 γ, β。AdaLN [7] 让它们依赖条件 c，例如扩散时间步、类别或文本表示：

$$\mathrm{AdaLN}(x, c) = (1 + s(c)) \odot \mathrm{LN}_0(x) + b(c), \qquad [s(c), b(c)] = \mathrm{MLP}(c). \tag{2.22}$$

这里 LN_0 表示无仿射参数的 LN。条件网络从 c 生成 $s(c)$ 和 $b(c)$，分别调整归一化特征的幅度与偏移。写成 $1 + s(c)$ 后，当 $s(c) = 0$ 时缩放系数就是 1，便于初始化。

AdaLN-Zero 还为残差分支引入条件门：

$$y = x + \alpha(c) \odot F\big((1 + s(c)) \odot \mathrm{LN}_0(x) + b(c)\big)\,. \tag{2.23}$$

若初始化时 $\alpha(c) = 0$，则该子块满足 $y = x$。一种实现是将生成 s, b, α 的最后一层置零，让新加的残差变换从接近恒等映射开始。

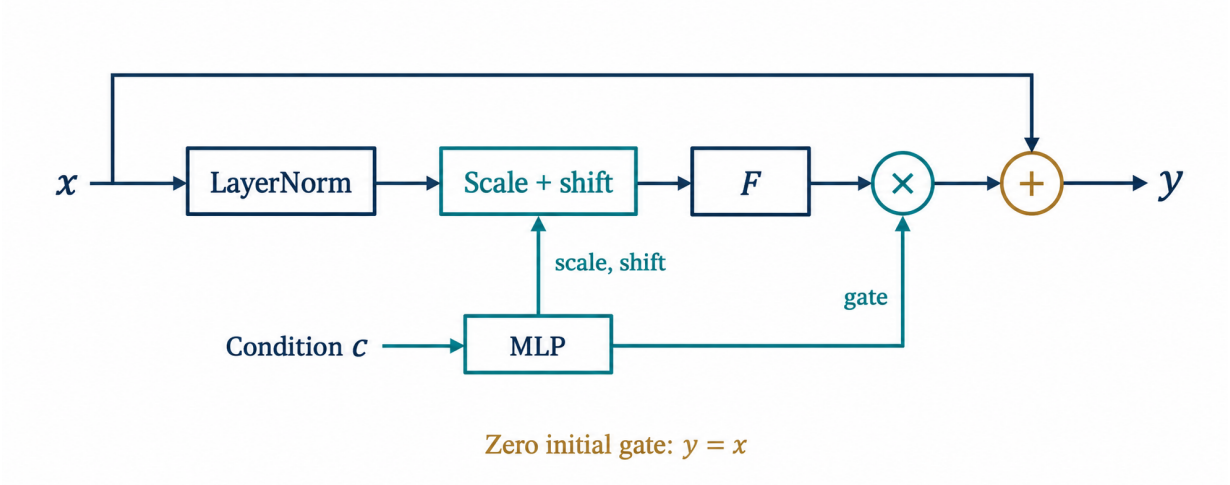


图 2.3 一个简化的 AdaLN 残差子块。条件分支同时生成调制量和残差门；完整 Transformer 块通常分别调制注意力与 FFN。

数学直觉 | AdaLN 与文本条件

AdaLN 可以先把文本编码为条件向量，再由条件网络生成每个通道的缩放与偏移。这样，同一个图像特征在不同文本条件下会得到不同的处理，文本通过这些参数影响后续计算。

AdaLN-Zero 的实现

算法实现 | AdaLN-Zero

```
class AdaLNBlock(nn.Module):
    def __init__(self, d, cond_dim):
        super().__init__()
        self.norm = nn.LayerNorm(d, elementwise_affine=False)
        self.mod = nn.Linear(cond_dim, 3 * d)
        self.fn = nn.Sequential(nn.Linear(d, 4 * d),
                                nn.GELU(), nn.Linear(4 * d, d))
        nn.init.zeros_(self.mod.weight)
        nn.init.zeros_(self.mod.bias)

    def forward(self, x, c):      # x: [B,T,d]; c: [B,cond_dim]
        s, b, a = self.mod(c).chunk(3, dim=-1)
        u = (1 + s[:, None]) * self.norm(x) + b[:, None]
        return x + a[:, None] * self.fn(u)
```

该实现对应式 (2.23)，包含一个 FFN 残差分支。$[B, d]$ 的条件参数沿 token 维广播成 $[B, 1, d]$。

初始化时，将生成 s, b, α 的末层置零，残差门 α 就从零开始。残差分支 F 保持正常初始化，使它能够产生用于更新门的信号。

更精确地，令 $u = (1 + s) \odot \mathrm{LN}_0(x) + b$。在初始化 $\alpha = 0$ 时，对分支参数 θ_F 有

$$\frac{\partial y}{\partial \theta_F} = 0, \qquad \frac{\partial y}{\partial \alpha} = \mathrm{diag}(F(u)). \tag{2.24}$$

第一步中，残差分支参数的梯度为零，门参数仍可以通过 $F(u)$ 获得梯度。门更新后，残差分支开始接收到梯度，并逐渐参与输出计算。

> **备注 ｜ 归一化方法的选择**
> 选择归一化方法时，可以先写出输入形状和计算统计量的维度。接着根据训练设置判断：batch 是否足够大，均值是否包含有用信息，模型是否需要文本等外部条件。这些问题分别对应 BN、LN ／ RMSNorm 和 AdaLN 的使用特点。

2.4 损失函数与优化

训练通过损失函数衡量预测与目标之间的差距，再调整参数以减小损失。监督学习常写为

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell(f_\theta(x_i), y_i) + \lambda R(\theta). \tag{2.25}$$

其中 ℓ 衡量单个样本的预测误差，$R(\theta)$ 是正则项。选择损失函数时，可以先说明输出的概率模型，再推导相应的负对数似然。

回归与分类损失

若假设 $y \mid x \sim \mathcal{N}(f_\theta(x), \sigma_y^2 I)$，并固定噪声方差，那么负对数似然中与参数有关的部分正比于平方误差。这给出了 MSE 的一种概率解释，适合用高斯噪声描述误差的回归模型。

对二分类，令 logit 为 $z \in \mathbb{R}$、$p = \sigma(z)$，则

$$\ell_{\mathrm{BCE}}(z, y) = -y \log p - (1-y)\log(1-p) \tag{2.26}$$

$$= \mathrm{softplus}(z) - yz, \qquad y \in \{0, 1\}. \tag{2.27}$$

对 K 类互斥分类，令 $z \in \mathbb{R}^K$，$p_k = e^{z_k} / \sum_j e^{z_j}$，则

$$\ell_{\mathrm{CE}}(z, y) = -\log p_y = \log \sum_{k=1}^{K} e^{z_k} - z_y. \tag{2.28}$$

实现时用 log-sum-exp 技巧：取 $m = \max_k z_k$，把对数配分函数改写为 $m + \log \sum_k e^{z_k - m}$，避免指数溢出。

> **数学直觉 ｜ 交叉熵的梯度**
> 对 one-hot 目标向量 q，有 $\partial \ell / \partial z_k = p_k - q_k$。预测概率超过目标的类被压低，低于目标的类被抬高。对固定目标分布 q，交叉熵 $H(q, p) = H(q) + D_{\mathrm{KL}}(q \| p)$，因此最小化交叉熵也在缩小对应 KL 散度。

序列损失与多任务损失

对于第一章的条件文本生成，负对数似然分解为

$$\mathcal{L}_{\mathrm{gen}} = -\frac{1}{\sum_{i,t} m_{it}} \sum_{i,t} m_{it} \log p_\theta(w_{it} \mid w_{i,<t}, v_i), \tag{2.29}$$

其中 $m_{it} \in \{0, 1\}$ 表示该 token 是否参与损失计算。例如，将 padding 的掩码设为零，就只统计实际文本。上式对所有有效 token 求平均，因此较长的样本贡献更多项；若先分别计算每个样本的平均损失，再对样本平均，则各样本权重相同。

算法实现 | 损失函数的实现

```
loss_cls = F.cross_entropy(logits, labels)     # [B,K], [B]
loss_bin = F.binary_cross_entropy_with_logits(z, target)
loss_reg = F.mse_loss(pred, target_reg)
```

前两行直接接收模型输出的 logits，函数内部完成所需的概率变换与对数计算。互斥类别使用 softmax 交叉熵；一个样本可以同时拥有多个标签时，通常对各类别分别使用 BCE。

多模态训练还可能组合多项损失，例如

$$\mathcal{L} = \lambda_{\text{con}}\mathcal{L}_{\text{contrastive}} + \lambda_{\text{gen}}\mathcal{L}_{\text{gen}} + \lambda_{\text{match}}\mathcal{L}_{\text{matching}}. \tag{2.30}$$

对比损失学习图文配对，生成损失训练文本输出，匹配损失判断两个输入是否对应。系数 λ 会同时缩放各项损失及其梯度，用来调整这些任务在训练中的比重。

备注 | 多任务的梯度

每个任务都会对共享参数产生梯度。两个梯度方向接近时，更新可以同时改善两个任务；方向相反时，则需要在两者之间折中。训练中可以结合各项梯度的大小与验证结果，调整损失权重。

小批量梯度下降

对参数 θ_t，最基本的更新为

$$\theta_{t+1} = \theta_t - \eta_t g_t, \qquad g_t = \frac{1}{B}\sum_{i\in\mathcal{B}_t}\nabla_\theta \ell_i(\theta_t). \tag{2.31}$$

均匀抽样且按样本平均损失时，mini-batch 梯度是全数据梯度的无偏估计。每个批次包含的样本不同，梯度也随之波动。增大 batch 通常会减小这种波动，同时增加单步计算量；训练效果可以通过验证集比较。

数学直觉 | 学习率与收敛

对一维二次函数 $L(w) = \frac{1}{2}aw^2$，$a > 0$，梯度下降满足 $w_{t+1} = (1-\eta a)w_t$。只有 $|1-\eta a| < 1$，即 $0 < \eta < 2/a$，迭代才收敛。曲率越大，稳定步长越小；多维问题中最陡方向会约束统一学习率。

动量、Adam 与学习率调度

动量把多个步骤的梯度方向积累起来。一种常用约定是

$$v_t = \beta v_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta_t v_t. \tag{2.32}$$

连续几步都朝同一方向的梯度会积累起来，来回变化的部分则会相互抵消。有些实现对新梯度再乘以 $1-\beta$，因此比较动量公式时，需要连同学习率一起看。

Adam 进一步估计梯度的一阶、二阶矩：

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2, \tag{2.33}$$

$$\hat{m}_t = m_t/(1-\beta_1^t), \qquad \hat{v}_t = v_t/(1-\beta_2^t), \tag{2.34}$$

$$\theta_{t+1} = \theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon}. \tag{2.35}$$

平方、开方和除法都逐元素进行。二阶矩估计较大的坐标会得到较小的有效步长，所有坐标再共同乘以全局学习率 η_t。由于 m_0, v_0 初始化为零，训练初期的矩估计偏小，带帽的量对这一偏差作了修正。

学习率衰减帮助模型在后期用较小步长细化解。阶梯衰减可写成 $\eta_t = \eta_0 \rho^{k(t)}$，其中 $k(t)$ 为截至当前已发生的衰减次数，$0 < \rho < 1$。

一个线性 warmup 加余弦衰减的例子为

$$\eta_t = \begin{cases} \eta_{\max}\, t/T_w, & 0 \le t < T_w, \\ \eta_{\min} + \dfrac{\eta_{\max} - \eta_{\min}}{2}\left[1 + \cos\left(\pi \dfrac{t - T_w}{T - T_w}\right)\right], & T_w \le t \le T. \end{cases} \tag{2.36}$$

T_w 为预热步数，T 为总步数。前 T_w 步逐渐提高学习率，之后沿余弦曲线降低它。这样，训练开始时使用较小步长，后期也能用小步长细化参数。预热长度和衰减速度可以根据训练进展调整。

> **备注 | 学习率调整**
> 损失进入平台期时，先查看当前学习率和梯度范数，再检查模型是否仍能拟合小批样本。较大的步长可能使参数在低损失区域附近来回移动，较小的步长则可能使训练进展缓慢。结合这些观察，每次调整少量设置，便于判断变化的作用。

2.5 正则化与训练

L2、L1 与权重衰减

L2 正则采用 $R(\theta) = \frac{1}{2}\|\theta\|_2^2$。对普通 SGD，

$$\theta_{t+1} = \theta_t - \eta_t(\nabla \mathcal{L}_{\text{data}} + \lambda \theta_t) = (1 - \eta_t \lambda)\theta_t - \eta_t \nabla \mathcal{L}_{\text{data}}. \tag{2.37}$$

把更新式整理后可以看到，普通 SGD 在每一步都将原参数乘以 $1 - \eta_t\lambda$，因此 L2 正则在这里等价于乘性权重衰减。Adam 会先用梯度更新矩估计；若把 $\lambda\theta_t$ 加入梯度，这一项也会参与自适应缩放。

AdamW 将权重衰减单独执行，再进行自适应梯度更新：[8]

$$\theta_{t+1} = (1 - \eta_t\lambda)\theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \tag{2.38}$$

实践中常对不同参数组设置不同衰减，例如对偏置或归一化参数不施加衰减；应与所复现的训练配置一致。[8]

从最大后验估计看，若参数服从零均值高斯先验，负对数先验产生 L2 惩罚；若服从独立 Laplace 先验，则产生 L1 惩罚 $\lambda\|\theta\|_1$。这说明正则项是在数据拟合之外表达对参数的偏好。

> **数学直觉 | L2 与 L1 正则**
> 在 $w_1 + w_2 = 1$ 下，$w_1^2 + w_2^2$ 在 $(1/2, 1/2)$ 处最小，说明 L2 倾向于把权重分散到两个分量。L1 的稀疏作用则可以通过下面的一维优化问题看出来：它在零点有尖角，使一段范围内的输入都对应零解。

例如，对 $\frac{1}{2}(w - u)^2 + \lambda|w|$，最优解是软阈值

$$w^* = \operatorname{sign}(u)\max(|u| - \lambda, 0). \tag{2.39}$$

当 $|u| \le \lambda$ 时，最优解恰好为零；超过这一范围后，解的绝对值比 $|u|$ 小 λ。这种软阈值运算说明了 L1 如何产生稀疏参数。

Dropout

设丢弃概率为 p，保留概率为 $q = 1 - p > 0$，$m_j \sim \mathrm{Bernoulli}(q)$。常见的 inverted dropout 在训练时使用

$$\tilde{h}_j = \frac{m_j}{q} h_j, \qquad \mathbb{E}[\tilde{h}_j \mid h_j] = h_j, \qquad \mathrm{Var}(\tilde{h}_j \mid h_j) = \frac{p}{q} h_j^2. \tag{2.40}$$

推理时直接使用 h。这种缩放保持单层激活的条件期望，但噪声强度会随丢弃率和激活幅值增加。

> **数学直觉 | Dropout 的子网络解释**
>
> 每个随机掩码选择一部分单元参与计算，这些不同的子网络共享同一组权重。训练使权重适应单元随机缺失的情况。推理时使用完整网络和训练时约定的缩放；对含非线性的网络，通常有 $\mathbb{E}[f(\tilde{h})] \neq f(\mathbb{E}[\tilde{h}])$，两种平均方式会得到不同结果。

> **算法实现 | 训练模式与验证模式**
>
> ```
> model.train()                    # dropout on; BN training mode
> loss = criterion(model(x), y)
>
> model.eval()                     # dropout off; BN eval mode
> with torch.no_grad():
>     pred = model(x_val)
> ```
>
> `eval()` 将 dropout、BN 等层切换到推理方式，`no_grad()` 关闭该段计算的梯度记录。验证时通常同时使用二者。[9]

数据增强通过变换输入增加训练样本的变化。若变换 a 保持标签含义，训练目标可以写为

$$\mathbb{E}_a[\ell(f_\theta(a(x)), y)].$$

在图文任务中，图像与文字需要一起考虑，例如翻转图像后，描述中的“左”和“右”也要相应调整。

梯度过大时，可以在更新参数前限制其范数。对梯度 g，一种常见做法为

$$g \leftarrow g \min\left(1, \frac{c}{\|g\|_2 + \epsilon}\right) \tag{2.41}$$

当梯度范数超过阈值 c 时，这个式子按比例缩小整个梯度，保持方向并限制幅度。混合精度训练应先取消梯度缩放，再执行裁剪。

微调多模态模型时，可以先冻结编码器，只训练新加入的连接层，再用较小学习率联合训练。冻结参数后，BN 的运行统计量是否继续更新，仍由模块的训练模式控制。

训练检查

开始完整训练前，可以先用少量样本检查损失、梯度和参数更新。确认这些步骤正常后，再增加数据量并比较训练设置。

算法实现 | 分类训练循环

```
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4,
                              weight_decay=1e-2)
for epoch in range(num_epochs):
    model.train()
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad(set_to_none=True)
        loss = F.cross_entropy(model(x), y)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()

    model.eval()
    with torch.no_grad():
        # evaluate on validation data; save the best checkpoint
        val_metric = evaluate(model, val_loader)
```

示例省略数据与 `evaluate` 的定义，假设模型已移到 `device`。学习率与正则强度需按任务选择；该示例对全部参数使用统一衰减。

1. **检查初始损失**。若 K 类预测近似均匀，交叉熵应接近 $\log K$；二分类预测接近 $1/2$ 时，BCE 应接近 $\log 2$。可以用这些数值检查模型输出与标签是否对应。
2. **过拟合少量干净样本**。暂停强增强、dropout 和衰减，验证模型能否记住一个小批次。若不能，先查标签、形状、梯度、参数更新和容量。
3. **找可下降的学习率**。在短训练中确认损失显著下降，再恢复正则项。记录实际学习率，避免调度器使设定值与运行值不同。
4. **先粗后细搜索**。围绕可行区间比较少量配置，根据验证集选择候选并延长训练；测试集留到最终评估。
5. **同时看训练与验证**。记录分项损失、任务指标与梯度状态。训练集与验证集比较时，尽量使用相同的评价模式和预处理。

多模态训练的数据检查

图文训练还需检查样本配对、mask 和特征尺度。对比学习的相似度矩阵中，对角线通常放置真实配对。因此，数据打乱或跨设备收集后，要保持图像与文本索引的对应顺序。

备注 | 实验记录

实验记录包括数据划分、预处理、模型版本和随机种子，以及优化器、学习率调度和损失平均方式。保存验证集表现最好的检查点。使用 BN、dropout 或冻结模块时，也记录各模块采用的训练或推理模式。

观测	可能原因	优先检查
损失变为 NaN ／ Inf	数值溢出、步长过大、非法数据	首个非有限张量、loss 输入、混合精度与梯度
训练损失几乎不降	参数未更新、梯度弱、步长不合适	小样本过拟合、梯度是否存在、优化器参数组
训练改善，验证恶化	过拟合、分布差异或数据泄漏问题	数据划分、早停、增强与正则
训练与验证都较差	欠拟合、优化不足、标签／指标问题	模型容量、训练时长、目标是否合理
两者接近且都很好	泛化可能良好	确认无泄漏与评价独立性

表 2.3 常见训练现象及其检查方法。可以结合损失曲线、梯度和小样本实验逐步定位原因。

思考题

1. 能否用两行公式说明：仿射层堆叠为什么不能替代非线性?
2. 给定 $[B, T, d]$，能否写出 SwiGLU、LN 与 RMSNorm 的中间形状?
3. 为什么 AdaLN-Zero 可以初始化为恒等映射，而门仍能开始学习?
4. 为什么普通 SGD 下的 L2 与权重衰减等价，而 Adam 下通常不等价?
5. 新增模态后精度没有提升，如何区分数据、结构和优化三类原因?

第 3 章　计算机视觉

计算机视觉研究怎样从图像和视频中识别物体、判断位置、理解场景。图像保存为像素数组，模型需要从这些数值中提取有用的特征。例如，局部的明暗变化可以形成边缘，边缘和纹理又可以组合成物体的部件。在多模态模型中，视觉编码器先完成这些计算，再把得到的特征交给后续的图文模型。

第二章介绍了神经网络的一般计算和训练方法。本章将它们用于图像。图像中的相邻像素通常有关联，同一种边缘或纹理也可能出现在不同位置。卷积神经网络（convolutional neural network，CNN）利用这些特点安排连接方式，让局部特征的提取更有效率。

3.1　卷积

局部连接与参数共享

设一幅图像为 $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$。若直接展平后输入含 M 个输出单元的全连接层，仅权重就需要 $MC_{\text{in}}HW$ 个参数。例如，单通道 200×200 图像连接到 40 000 个单元，需要 1.6×10^9 个权重。这种连接允许任意位置之间发生作用，却没有优先利用图像的空间结构。

第一步改进是**局部连接**：每个输出位置只观察附近的一个小窗口。如果每个单元连接一个 10×10 的单通道窗口，40 000 个输出单元需要 4×10^6 个权重。这时，各位置仍分别学习自己的参数，例如图像左侧和右侧的边缘检测器需要分别训练。

第二步改进是**参数共享**：让同一种局部检测规则在所有位置重复使用。上面的 10×10 窗口此时只需 100 个共享权重，即一个卷积核；使用 20 个这样的核则需要 2 000 个权重，另加每个输出通道的偏置。这些比较忽略偏置，且假定输出位置数保持不变，以便单独观察连接方式的影响。

参数共享让同一种局部检测规则可以在不同位置使用。例如，检测竖直边缘的核可以沿整幅图像移动。真实图像中，不同位置仍有各自的特点，例如天空常出现在上方；卷积主要利用边缘、纹理等局部模式可以重复出现这一规律。

卷积核与输出通道

先考虑步长为 1、没有填充的普通二维卷积。令卷积核大小为 $k_h \times k_w$，权重张量为

$$\mathcal{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w}, \qquad b \in \mathbb{R}^{C_{\text{out}}}.$$

采用从 0 开始的索引，第 o 个输出通道在位置 (i, j) 的值为

$$Y_{o,i,j} = b_o + \sum_{c=0}^{C_{\text{in}}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} \mathcal{W}_{o,c,u,v} X_{c,i+u,j+v}. \tag{3.1}$$

因此，一次局部计算同时汇集空间邻域和全部输入通道。一个 5×5 的 RGB 卷积核实际包含 $3 \times 5 \times 5$ 个权重，产生一个标量响应；把它移遍图像，得到一张特征图。使用多个核，就得到多个输出通道。

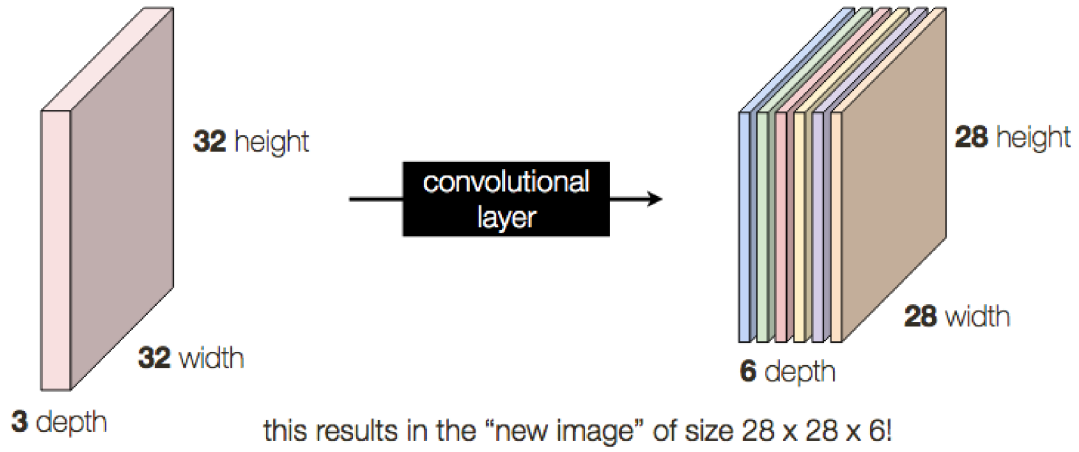


图 3.1 六个卷积核分别产生六张特征图。输入按 $[C, H, W]$ 记为 $[3, 32, 32]$，使用 5×5 核、步长 1、无填充后，输出为 $[6, 28, 28]$。图中的 depth 指通道维度。

在这个例子中，每个输出通道有 $3 \times 5 \times 5 + 1 = 76$ 个参数，整层共有 $6 \times 76 = 456$ 个参数。每个核在 28×28 个位置上重复使用，产生相应数量的响应。输出通道表示不同核提取的特征，越往后的层，其特征含义越由训练任务决定。

卷积核可以具有直观的作用。例如，元素均为 $1/9$ 的 3×3 核计算局部均值，使图像平滑；核

$$\begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}$$

比较窗口左右两侧的亮度，对竖直边缘产生较强响应。CNN 中的核通常由任务损失学习，无需事先指定为边缘检测器。

> **备注 | 卷积与互相关**
>
> 式 (3.1) 采用互相关的写法，直接将核与对应窗口相乘并求和。PyTorch 的 `Conv2d` 也使用这一约定。[10] 数学上的卷积还会先翻转核。核由训练自由学习时，两种约定具有相同的表达范围；手工设置方向性滤波器时，则按所用约定排列核的元素。

步长、填充与输出尺寸

步长 s 指相邻窗口起点之间的距离，填充 p 指在输入两侧各补入的宽度。进一步用 δ 表示膨胀率，即核内相邻采样点的间隔，大小为 k 的核覆盖的有效跨度为

$$k_{\text{eff}} = \delta(k - 1) + 1. \tag{3.2}$$

普通卷积取 $\delta = 1$。若高、宽方向采用对称填充，输出尺寸为

$$H_{\text{out}} = \left\lfloor \frac{H + 2p_h - \delta_h(k_h - 1) - 1}{s_h} \right\rfloor + 1, \quad W_{\text{out}} = \left\lfloor \frac{W + 2p_w - \delta_w(k_w - 1) - 1}{s_w} \right\rfloor + 1. \tag{3.3}$$

这个公式来自窗口起点的计数。以高度为例，最后一个合法起点不能超过 $H + 2p_h - k_{\text{eff}}$；从 0 起每次前进 s_h，就得到式中的向下取整和末尾的 $+1$。这里假定填充后的输入足以容纳至少一个窗口。

例如，7×7 输入使用 3×3 核、步长 3、无填充，输出为 2×2。窗口起点分别位于 0 和 3，边缘剩余位置不足以再容纳一个完整窗口，因此使用向下取整。

对奇数核、步长 1 和膨胀率 1，取 $p=(k-1)/2$ 可以保持高、宽不变。步长大于 1 时，需要根据期望输出尺寸重新计算填充量。

填充可以在保留分辨率的同时继续提取特征，并让边界附近的像素参与更多次计算。零填充把图像外部的值设为零，因此边界处的卷积窗口会同时包含图像像素和填充值。

平移等变性与 1×1 卷积

令 T_Δ 表示整数平移。对于无限网格上的步长 1 卷积 $\mathcal{C}$，参数共享给出

$$\mathcal{C}(T_\Delta X)=T_\Delta \mathcal{C}(X). \tag{3.4}$$

输入中的边缘移动后，特征响应也移动相同的距离，这称为**平移等变性**。平移不变性则指输入移动后，输出保持原样。上式适用于无限网格上的步长 1 卷积；有限图像还会受到边界填充的影响。使用较大步长时，需要让输入平移与采样网格对齐，才能保留对应关系。

当核为 1×1、步长为 1 时，式 (3.1) 变成

$$Y_{:,i,j}=AX_{:,i,j}+b,\qquad A\in\mathbb{R}^{C_{\text{out}}\times C_{\text{in}}}. \tag{3.5}$$

这个计算在每个位置独立地混合通道，可以把通道数压缩或扩展。两个 1×1 卷积之间加入非线性激活，就形成逐位置的前馈网络，与第二章的逐 token FFN 相似。相邻位置之间的信息交换则由空间卷积等操作完成。

3.2 池化、感受野与分类

识别物体时，网络需要把边缘、纹理等局部特征组合起来。为此，后续层会逐渐观察更大的图像区域，同时减少特征图中的位置数。本节分别介绍这种空间压缩，以及每个输出位置能观察到多大范围的输入。

池化

池化通常分别作用于每个通道。对于位置 (i,j) 对应的窗口 $\Omega_{i,j}$，最大池化与平均池化分别为

$$Y^{\max}_{c,i,j}=\max_{(u,v)\in\Omega_{i,j}} X_{c,u,v},\qquad Y^{\text{avg}}_{c,i,j}=\frac{1}{|\Omega_{i,j}|}\sum_{(u,v)\in\Omega_{i,j}} X_{c,u,v}. \tag{3.6}$$

最大池化保留局部最强响应，平均池化汇总窗口中的整体响应。两者通常没有可学习参数，也不改变通道数。2×2 窗口配合步长 2，在尺寸为偶数时把高、宽各减半，输出元素数变为原来的四分之一。

池化把一个窗口压缩成一个数，所以输出保留了较粗的位置。强响应在同一窗口内小幅移动时，最大值可能保持不变；跨过窗口边界后，它就可能出现在另一个输出位置。因而池化可以容忍部分局部位移。对小物体或细窄边界，下采样过早则容易丢失定位所需的细节。

反向传播时，最大池化将梯度传给取得最大值的位置，并列最大值的处理由实现决定。平均池化按平均系数向窗口内的位置分配梯度。这两种方式保留的信息不同，梯度经过的位置也不同。

感受野

感受野描述一个输出单元可能依赖的输入区域。沿一条由局部算子构成的串行路径，令 r_ℓ 为第 ℓ 层感受野的跨度，j_ℓ 为相邻输出位置在原图上的间隔。初始化 $r_0 = j_0 = 1$，递推为

$$j_\ell = j_{\ell-1} s_\ell, \qquad r_\ell = r_{\ell-1} + (k_{\text{eff},\ell} - 1) j_{\ell-1}. \tag{3.7}$$

高、宽方向可以分别计算。递推式给出感受野的跨度，填充决定它在输入上的位置。对于边界处的输出，一部分感受野可能落在填充区域；对于膨胀卷积，跨度内的实际采样位置之间还可能有空隙。

三个连续的 3×3、步长 1 卷积，其感受野依次为 3×3、5×5、7×7。若在两个卷积之间插入一次步长 2 的池化，后续卷积的一步就对应原图中的两个位置，感受野会增长得更快。

操作	核与步长	j_ℓ	r_ℓ
输入	—	1	1
第一次卷积	$k=3,\ s=1$	1	3
第二次卷积	$k=3,\ s=1$	1	5
最大池化	$k=2,\ s=2$	2	6
第三次卷积	$k=3,\ s=1$	2	10

表 3.1 沿单个空间方向追踪感受野。池化以后，新增一个相邻特征位置对应原图中更大的间隔。

> **数学直觉 | 理论感受野与有效感受野**
> 理论感受野列出了可能影响某个输出的输入位置。实际模型对这些位置的依赖程度有强有弱，可以用输入梯度观察这种差异，得到有效感受野。对于包含全局池化或动态权重的网络，还需要沿相应的计算过程分析依赖范围。

分类头

用于图像分类的 CNN 通常把特征提取部分称为主干网络（backbone），再由分类头把特征转换为 K 个类别的 logits。若最终特征为 $U \in \mathbb{R}^{C \times H' \times W'}$，展平后连接全连接层需要 $KCH'W' + K$ 个参数；若先做全局平均池化（GAP），则

$$z_c = \frac{1}{H'W'} \sum_{i,j} U_{c,i,j}, \qquad a = W_{\text{cls}} z + b_{\text{cls}}, \quad W_{\text{cls}} \in \mathbb{R}^{K \times C}. \tag{3.8}$$

此时分类层只有 $KC + K$ 个参数。全局平均池化将每个通道汇总为一个数，因此在主干支持的尺寸范围内，分类头可以接收不同大小的特征图。这种表示适合整图分类。区域定位和视觉问答还需要知道特征出现在哪里，可以继续保留空间特征图供后续处理。

训练时，把 a 交给第二章的交叉熵函数。评估时，top-1 准确率检查最高分类别是否正确，top-5 准确率检查真实类别是否在前五个候选中。常用的 ImageNet 分类任务使用 1 000 类子集。比较模型时应采用相同的数据划分、输入分辨率，以及单模型或集成设置。

算法实现 | CNN 分类器

```
import torch
from torch import nn

net = nn.Sequential(
    nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
    nn.AdaptiveAvgPool2d(1), nn.Flatten(1),
    nn.Linear(64, 10),
)
x = torch.randn(4, 3, 32, 32)
logits = net(x)                    # [4, 10]
```

批量图像使用 $[B, C, H, W]$ 布局。第二个卷积后的特征为 $[4, 64, 16, 16]$，全局平均池化将它变为 $[4, 64, 1, 1]$，展平后再由线性层生成 10 个 logits。交叉熵函数可以直接接收这些 logits。

3.3 CNN 的计算量与结构

局部连接和参数共享减少了网络规模，深层 CNN 的训练仍需要较多资源。分析资源开销时，要分别计算参数量、中间激活占用和运算次数。它们通常集中在不同的层，也对应不同的改进办法。

参数量、显存与计算量

对普通卷积，设 $k_h = k_w = k$，不计激活函数等额外运算，则

$$P = C_{\text{out}} C_{\text{in}} k^2 + C_{\text{out}}, \tag{3.9}$$

$$A = B C_{\text{out}} H_{\text{out}} W_{\text{out}}, \tag{3.10}$$

$$\text{MACs} = B H_{\text{out}} W_{\text{out}} C_{\text{out}} C_{\text{in}} k^2. \tag{3.11}$$

P 包含偏置，A 是输出元素数。MAC 表示一次乘法及其累加；若把乘法和加法分别计作一次浮点运算，FLOPs 约为 2 MACs。一些资料把一组乘加计作一次，因此数值比较必须先统一口径。

以一个 AlexNet 风格的卷积层为例，输入为 $[3, 227, 227]$，输出通道数 64，核大小 11，步长 4，填充 2。按式 (3.3)，输出为 $[64, 56, 56]$，参数为 23 296 个，单张图像的计算量为 72 855 552 MACs。FP32 输出激活占 $64 \times 56 \times 56 \times 4 = 802\,816$ 字节，即 784 KiB。这组尺寸与通道配置用于计算示例，阅读其他 AlexNet 实现时仍应以具体配置为准。

同一网络末端若把 $[256, 6, 6]$ 特征展平为 9 216 维，再映射到 4 096 维，则需要

$$9\,216 \times 4\,096 + 4\,096 = 37\,752\,832$$

个参数。这个全连接层的输出只有 4 096 个数，权重却有三千多万个。卷积层的情况常常相反：同一组权重在许多位置重复使用，因此参数较少、运算次数较多。衡量模型开销时，需要分别查看这些数量。

训练时，显存同时保存参数、中间激活、梯度和优化器状态，算子还可能需要临时工作空间。提高输入分辨率后，中间激活和卷积计算都会增加，同一卷积层的参数量则保持不变。

VGG

VGG [11] 把卷积层组织成若干阶段：阶段内重复使用 3×3、步长 1、填充 1 的卷积，在阶段之间用 2×2、步长 2 的池化降低分辨率。VGG-16 的五个阶段分别含 $2,2,3,3,3$ 个卷积层，VGG-19 则为 $2,2,4,4,4$ 个，末端再接三个全连接层。

为什么反复使用小核？在输入、中间与输出通道均为 C、空间尺寸相同且忽略偏置时，一个 5×5 卷积有 $25C^2$ 个参数，而两个 3×3 卷积只有 $18C^2$ 个，并且两者具有相同的 5×5 理论感受野。三个 3×3 卷积以 $27C^2$ 个参数获得 7×7 感受野，也少于单个 7×7 卷积的 $49C^2$。

多个小卷积之间可以插入激活函数，使特征经过更多次非线性变换。前面的参数比较采用相同通道宽度和普通卷积。在这些条件下，小核堆叠既扩大了感受野，也减少了参数；换成逐通道卷积后，核大小与开销的关系还会改变。

VGG 的阶段通道数通常依次为 $64,128,256,512,512$。空间缩小后增加通道，能够让较少的位置携带更丰富的特征。若高、宽各减半而输入输出通道都加倍，式 (3.11) 中的空间因子约缩小四倍、通道因子约增大四倍，相应卷积层的计算量可以保持同一数量级。末端展平接大规模全连接层仍会带来很高的参数开销。

Inception

同一图像中，局部纹理和整体轮廓需要不同大小的观察范围。Inception [12] 在一个模块内并行使用多种大小的卷积和池化，再沿通道维拼接结果。各分支保持相同的高、宽；若通道数分别为 $C_1,\dots,C_4$，拼接后就有 $\sum_{m=1}^{4} C_m$ 个通道。

直接让 3×3 和 5×5 核处理所有输入通道会很昂贵。Inception 的做法是在这些分支前加入 1×1 卷积，把输入通道从 C 压到 R，再做空间卷积。对输出通道为 D 的 $k\times k$ 分支，忽略偏置时参数由 k^2CD 改为

$$CR + k^2RD. \tag{3.12}$$

当 R 足够小时，新增的投影开销小于后续卷积节约的开销。这个窄的中间表示称为瓶颈（bottleneck）；它通过学习决定哪些通道组合值得继续处理。

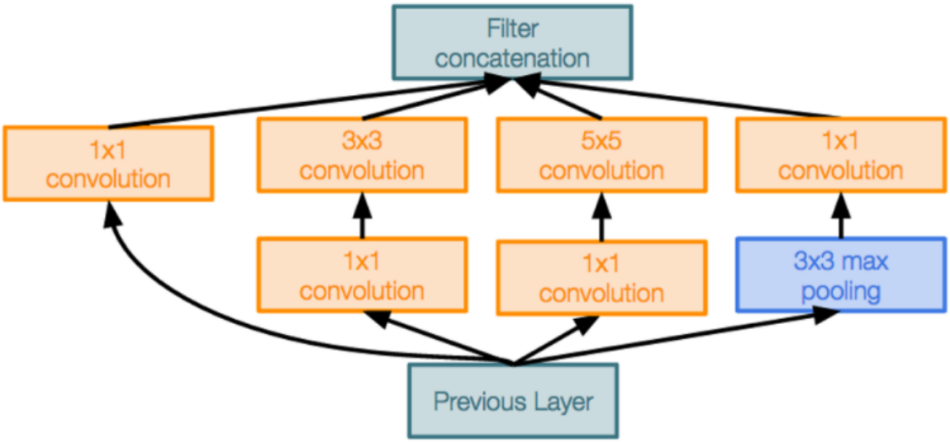


图 3.2 Inception 模块的分支结构 [12]。大核分支先用 1×1 卷积减少通道，池化分支后也可用 1×1 卷积控制输出宽度，最后沿通道维拼接。

例如，输入为 $[256,28,28]$，三个卷积分支的输出通道数分别为 $128,192,96$，池化分支保留 256 个通道，朴素拼接会产生 672 个通道，三个卷积分支约需 854.2M MACs。若把大核分支的中间通道压到 64，并把池化分支输出也投影到 64，输出变为 480 个通道，卷积计算约降至 271.4M MACs。

这里取单张图像，忽略池化、偏置与激活开销。瓶颈既节约当前层的计算，也避免通道数持续膨胀而增加后续代价。

GoogLeNet 将这样的模块堆叠成网络，末端采用全局平均池化以减少大规模全连接层。它还在中间位置加入辅助分类器，训练目标可写为 $\mathcal{L}_{\text{main}} + \sum_m \alpha_m \mathcal{L}_{\text{aux},m}$，为较早的层提供额外监督；推理时可以去掉这些辅助头。网络的深度、分支宽度和监督路径由此成为可以分别调整的设计因素。

3.4 ResNet 与 SENet

增加网络层数后，有时会观察到训练误差反而升高，说明新网络更难优化。ResNet 为每个块增加一条直接传递输入的通路，帮助深层网络保留已有特征。SENet 则根据当前图像，为不同特征通道计算各自的权重。

ResNet

假设一个块希望学习映射 $H(x)$。残差参数化把它写为 $H(x) = x + F(x)$，让可学习分支负责相对于输入的变化。若额外的块暂时没有必要改变表示，只需使 $F(x)$ 接近零，就可以保留输入。这比要求多层带非线性的普通网络自行学出恒等映射更直接。

原始 ResNet [13] 的常见块在相加后还有一次 ReLU，可写成

$$y = \mathrm{ReLU}(F(x) + S(x)), \tag{3.13}$$

其中 S 是捷径分支。两条分支形状相同时，可以取 $S(x) = x$。需要改变分辨率或通道数时，常用带相应步长的 1×1 卷积投影，使两边的形状一致后再逐元素相加。

对相加前的 $x + F(x)$，Jacobian 为 $I + J_F$，其中 I 对应直接传递输入的通路。式 (3.13) 在相加后还有 ReLU，因此完整的导数还要乘以激活的局部导数。如果 $F = 0$ 且输入非负，整个块就保留原输入。

浅层 ResNet 常用两个 3×3 卷积组成基本块。更深的版本采用 1×1、3×3、1×1 的瓶颈块，先减少通道，在较窄空间内完成空间卷积，再恢复输出宽度。它与 Inception 中瓶颈的共同点是降低昂贵算子的输入宽度；残差捷径则使块的整体输入输出仍能在合适的维度上相加。

> **数学直觉 | 残差块的展开**
>
> 忽略块末端激活，并假定维度始终一致，连续残差块满足
>
> $$x_L = x_0 + \sum_{\ell=0}^{L-1} F_\ell(x_\ell).$$
>
> 最终特征由初始输入和各块的修正量相加而成。第 ℓ 个修正量使用前面已经更新过的 x_ℓ，所以这些块依次配合，逐步改变表示。

SENet

不同图像可能需要不同的特征。例如，一张图像主要依赖纹理，另一张主要依赖轮廓。Squeeze-and-Excitation（SE）[14] 根据当前图像的特征计算通道权重，再用这些权重调整各通道的响应。

对特征 $U \in \mathbb{R}^{C\times H\times W}$，先通过全局平均池化把每个通道压缩为一个数，再由小型前馈网络生成门值：

$$z_c = \frac{1}{HW}\sum_{i,j} U_{c,i,j}, \tag{3.14}$$

$$s = \operatorname{sigmoid}(W_2\,\mathrm{ReLU}(W_1 z + b_1) + b_2)\,, \tag{3.15}$$

$$\widetilde{U}_{c,i,j} = s_c U_{c,i,j}. \tag{3.16}$$

若中间宽度为 C/r，则 $W_1 \in \mathbb{R}^{(C/r)\times C}$、$W_2 \in \mathbb{R}^{C\times(C/r)}$，忽略偏置时只增加约 $2C^2/r$ 个参数。这里假设 C/r 为整数，实际结构需明确取整后的宽度。

Squeeze 将每张特征图平均成一个数，得到整图的通道统计。Excitation 根据这些统计生成门值 $s_c \in (0,1)$，再把同一门值乘到该通道的所有位置。门值较小的通道受到较强抑制，门值较大的通道保留较多响应。各通道独立得到一个系数，彼此之间无需做和为 1 的归一化。

在残差网络中，可以让 SE 调节残差分支 $F(x)$，再与捷径相加。这样，恒等通路保留基础表示，门控决定当前图像需要怎样的特征修正。与第二章的 SwiGLU 一样，这里也利用乘法产生输入相关的调节，但 SE 的门来自空间汇总，作用于整张特征图的通道。

3.5 轻量化 CNN

移动设备的存储和计算资源有限，CNN 常需要缩小。可以先分析每一层的参数量、运算次数和数据传输，再决定减少通道、替换卷积方式，还是采用量化等方法。

深度可分离卷积

标准卷积中的每个输出通道同时组合所有输入通道及其空间邻域。深度可分离卷积（depthwise separable convolution）把这件事拆成两步：先对每个输入通道分别做空间卷积，再通过 1×1 卷积混合通道。第一步称为逐通道卷积（depthwise convolution），第二步称为逐点卷积（pointwise convolution）。

以每个输入通道只使用一个空间核、步长为 1 为例，省略偏置与非线性，有

$$Z_{c,i,j} = \sum_{u,v} D_{c,u,v} X_{c,i+u,j+v}, \qquad Y_{o,i,j} = \sum_{c} P_{o,c} Z_{c,i,j}. \tag{3.17}$$

逐通道卷积只移动空间信息，不交换通道信息；逐点卷积补上跨通道的线性组合。

在输入输出空间尺寸和通道数相同的比较条件下，标准卷积与这种分解的参数量分别为 $k^2 C_{\text{in}} C_{\text{out}}$ 和 $k^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}$。忽略偏置时，参数与 MACs 的比值都是

$$\frac{k^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}}{k^2 C_{\text{in}} C_{\text{out}}} = \frac{1}{C_{\text{out}}} + \frac{1}{k^2}. \tag{3.18}$$

例如 $k=3$、$C_{\text{in}}=64$、$C_{\text{out}}=128$ 时，参数从 73 728 减为 8 768，乘加约为原来的 11.9%。这是 MobileNet [15] 降低计算量的重要依据。

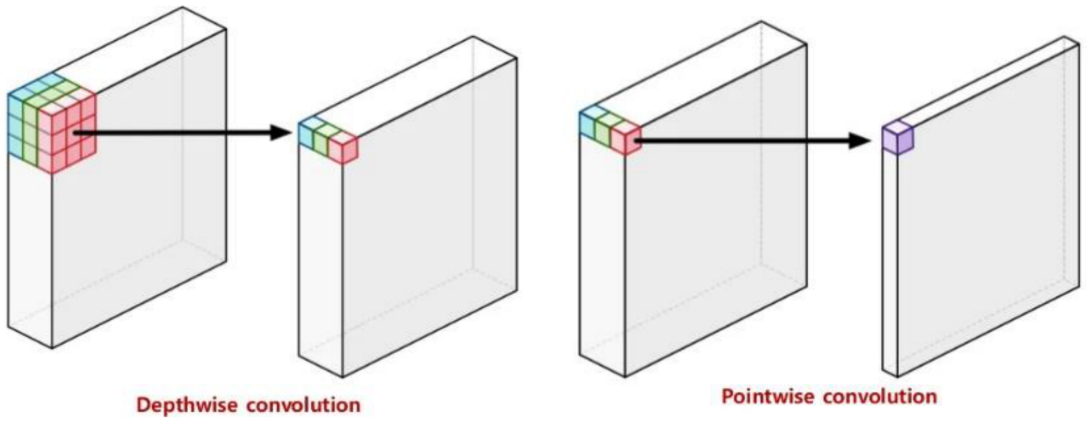


图 3.3 深度可分离卷积的两步计算 [15]。左侧分别处理各通道的邻域，右侧在同一空间位置混合通道。

> **数学直觉 ｜ 深度可分离卷积的等效核**
> 若两步之间没有非线性，把式 (3.17) 合并可得等效核
>
> $$\mathcal{W}_{o,c,u,v} = P_{o,c} D_{c,u,v}.$$
>
> 对固定输入通道 c，各输出通道使用同一个空间核 D_c，再分别乘以 $P_{o,c}$。标准卷积则允许每一对输入、输出通道使用不同的空间核。这种共享减少了计算，也缩小了单层可表示的核的范围。

PyTorch 的 `Conv2d(C,C,k,groups=C)` 可以实现逐通道卷积。它为每个通道设置一个空间核，权重形状为 $[C, 1, k, k]$。随后用 `Conv2d(C,D,1)` 将这 C 个通道组合成 D 个输出通道。

分组卷积与模型缩放

分组卷积将输入与输出通道分别划为 g 组，各组独立卷积，权重参数量为 $k^2 C_{\text{in}} C_{\text{out}}/g$。这里假设两种通道数都能被 g 整除。

如果连续几层保持同样分组，各组之间就很少交换信息。ShuffleNet 在两层之间重新排列通道，让下一层的每个组都接收到来自前面不同组的特征，再由卷积进行混合。

SqueezeNet 的 Fire 模块先用 1×1 卷积减少通道，再以并行的 1×1 和 3×3 分支提取特征，主要节省权重存储。GhostNet 则先计算一部分特征，再用较便宜的变换生成更多特征。

计算预算增加后，可以加深网络、增加通道，也可以提高输入分辨率。EfficientNet [16] 同时调整这三项，让网络的处理能力与输入细节一起增加。若深度、宽度和分辨率相对基线的倍率为 $\alpha^{\phi}, \beta^{\phi}, \gamma^{\phi}$，则对普通卷积占主导的网络，计算量粗略按

$$\text{Cost} \propto \alpha^{\phi} \beta^{2\phi} \gamma^{2\phi} = (\alpha \beta^2 \gamma^2)^{\phi} \tag{3.19}$$

增长。宽度同时影响输入和输出通道，所以近似以平方出现；分辨率也同时影响高、宽。这一估算帮助分配预算，具体缩放系数仍需通过验证选择。

剪枝减少连接或通道，量化降低数值位宽，知识蒸馏用教师模型帮助训练较小的学生。它们分别改变模型结构、数值表示和训练方式。实际速度还取决于硬件对稀疏计算、低位宽计算及各类卷积的支持。

例如，逐通道卷积的乘加很少，却仍需读取和写入特征图。数据传输占比较大时，运行速度的提升就会小于乘加次数的降幅。

神经架构搜索

上述设计都在选择核大小、通道宽度、分支和连接方式。神经架构搜索（neural architecture search，NAS）把这些选择编码成架构变量 a，在给定搜索空间中寻找性能较好的组合。一个理想化目标为

$$\min_{a} \ \mathcal{L}_{\mathrm{val}}(w^{*}(a), a) + \lambda\,\mathrm{Cost}(a), \qquad w^{*}(a) \in \arg\min_{w} \mathcal{L}_{\mathrm{train}}(w, a). \tag{3.20}$$

内部问题训练给定架构的权重，外部问题根据验证表现和资源代价比较架构。代价可以是参数量，也可以是在目标设备上测得的延迟；两种目标可能选出不同网络。

早期基于强化学习的 NAS [17] 用控制器产生候选网络。每个候选训练后，根据验证成绩得到奖励，再用奖励更新控制器。训练候选网络是这一过程的主要开销。

后续方法通过共享权重或缩短候选训练来降低成本。搜索完成后，再独立训练选中的结构，检查它在完整训练下的表现。搜索范围、计算预算和评价指标需要事先确定。

3.6 大核卷积与动态卷积

局部纹理有时不足以判断物体，需要结合更大范围的轮廓和场景信息。一种做法是增大卷积核，让每次计算覆盖更大的区域；另一种做法是根据整图内容生成卷积权重，使局部计算随图像变化。

大核逐通道卷积

VGG 比较的是相同通道宽度下的普通卷积。逐通道卷积的权重只有 $k^{2}C$ 个，大核的开销由此降低。例如 $C = 128$ 时，31×31 逐通道卷积有 123 008 个权重，而同宽度的 3×3 普通卷积有 147 456 个。逐通道卷积负责空间滤波，后续还需用逐点卷积等操作混合通道。

RepLKNet [18] 使用大核逐通道卷积汇集较大范围的空间信息，并配合残差连接和通道投影。它把空间滤波与通道混合分开安排，再通过专门的算子实现提高运行效率。

即使理论运算量可接受，硬件运行仍可能受访存限制。用 I 表示每传输一字节完成的浮点运算数，B_{mem} 表示内存带宽，P_{peak} 表示峰值计算吞吐，Roofline 模型给出的性能上界可写为

$$P_{\mathrm{attainable}} \le \min\{P_{\mathrm{peak}},\ B_{\mathrm{mem}} I\}. \tag{3.21}$$

当每次传输的数据只对应少量运算时，速度主要受内存带宽限制。提高数据复用、减少反复读写，可以让算子更有效地使用硬件。因此，实际延迟需要在目标设备上测量。

结构重参数化

大核的参数较多，优化也可能更困难。结构重参数化提供一种处理方式：训练时同时保留大核和小核分支，推理时把可以合并的线性分支折叠成一个核。

例如，大核与小核分支的采样网格一致，并在相加之前没有非线性，则

$$\mathcal{C}_{K_L}(X) + \mathcal{C}_{K_S}(X) = \mathcal{C}_{K_L + \mathrm{Pad}(K_S)}(X). \tag{3.22}$$

这里 $\mathrm{Pad}(K_S)$ 把小核居中补零到大核尺寸，各分支的偏置也应相加。这种合并保留了推理函数，同时让训练过程使用不同的参数化。

若分支中包含 BN，只有在使用固定统计量时才能先将它折叠进卷积。对某个输出通道，设卷积输出为 $u = Wx + b$，BN 的均值、方差为 μ, v，则

$$\widetilde{W} = \frac{\gamma}{\sqrt{v+\epsilon}} W, \qquad \widetilde{b} = \frac{\gamma(b-\mu)}{\sqrt{v+\epsilon}} + \beta. \tag{3.23}$$

各输出通道分别按此计算。折叠要求 BN 使用固定统计量，并且待合并的分支在相加前保持线性。训练态 BN 的统计量会随批次变化；包含 ReLU 的分支也有额外的非线性。这些情况需要保留相应计算。

大卷积核的作用

在 7×7 特征图上使用带填充的 13×13 核，可以让每个输出位置参考整张特征图。不同输出位置使用核中的不同偏移权重，因此仍会产生不同响应。这个设置的填充区域较大，边界处理也会明显影响结果。

RepLKNet 的消融实验中，扩大卷积核对语义分割的改善比对图像分类更明显。一种解释是，像素分类需要结合所在区域和周围场景，较大的邻域能够提供这些信息。这项结果来自论文给定的主干和训练设置，迁移到其他任务后可以再通过实验比较核大小。

核越大，独立权重通常越多。PeLK [19] 在核中心保留细粒度权重，在外围让较大区域共用参数，从而以较少的参数表示更大的核。普通卷积在不同图像位置复用整个核，PeLK 还让核内部的部分位置共享参数。它减少的是独立参数数量，运行时的稠密卷积仍需要相应计算。

OverLoCK

同一个局部轮廓，放在不同场景中可能有不同含义。OverLoCK [20] 先观察整幅图像，再用得到的场景信息帮助处理局部特征。

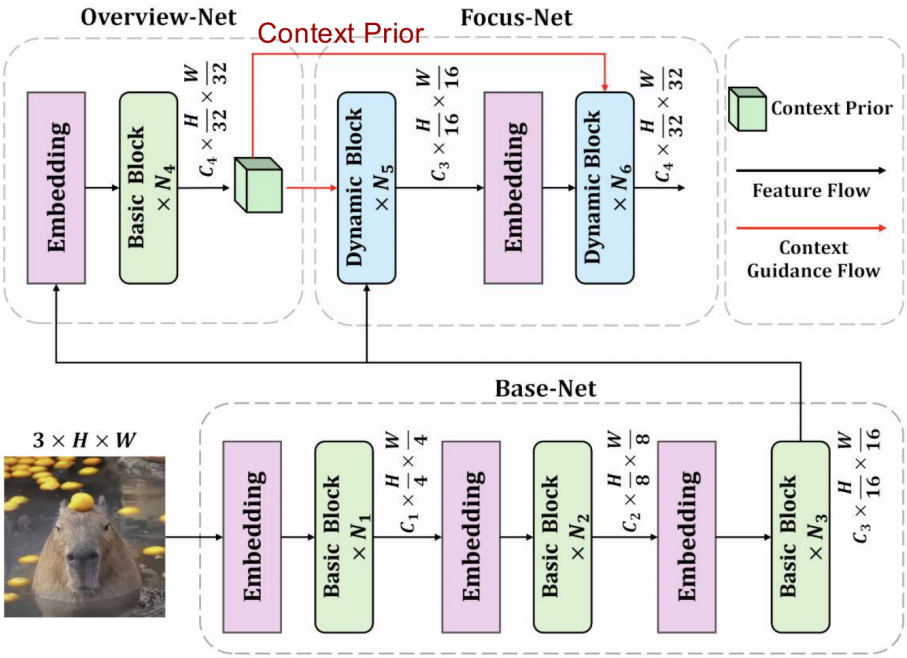


图 3.4 OverLoCK 的整体结构 [20]。Base-Net 的特征分别送入 Overview-Net 和 Focus-Net。红色箭头传递整图表示，黑色箭头传递各层特征。

它包含三个子网络。Base-Net 先提取低层和中层特征。Overview-Net 继续压缩这些特征，得到整幅图像的大致表示。Focus-Net 则同时接收中层特征和整图表示，进一步识别局部内容。

上下文既可以生成门值以调节特征，也可以直接参与卷积核的生成。其 ContMix 模块先将上下文汇总成 $S \times S$ 个区域中心，再计算每个位置与这些中心的相关性。省略分组下标，并采用位置为行的记法，令

$$Q \in \mathbb{R}^{HW \times d}, \qquad K \in \mathbb{R}^{S^2 \times d}, \qquad A = QK^\top \in \mathbb{R}^{HW \times S^2}.$$

Q 表示当前位置的投影，K 表示区域中心的投影；在受上下文指导的模块中，二者分别来自当前特征与上下文。随后将每一行相关性映射为局部核的权重：

$$D = \mathrm{softmax}_{\mathrm{row}}(AW_d), \qquad W_d \in \mathbb{R}^{S^2 \times k^2}, \quad D \in \mathbb{R}^{HW \times k^2}. \tag{3.24}$$

对输出位置 i，把 D 的第 i 行恢复为 $k \times k$ 核，并作用于它的局部邻域。以单个通道为例，核心计算可表示为

$$Y_{c,i} = \sum_{\Delta \in \Omega_k} D_{i,\Delta}(X, P) X_{c,i+\Delta}, \tag{3.25}$$

其中 P 表示上下文，Ω_k 是核内的空间偏移集合。完整模块采用分组设计，同组通道共享动态空间核，再配合投影、门控与残差结构。

式 (3.25) 将邻域内的特征加权求和，权重则由更广泛的上下文生成。因此，远处的内容可以通过权重影响当前区域的计算。SE 为每个通道生成一个系数，ContMix 则为每个位置生成空间核。

固定 S 和核大小时，相关性矩阵有 HWS^2 个元素，比所有位置两两比较的 $(HW)^2$ 更小。区域中心汇总了远处的信息，这种压缩也决定了模型能利用的上下文细节。

3.7 数据增强

即使网络能处理不同位置的物体，如果训练图像中的物体总在中央，它仍可能依赖固定位置。数据增强通过翻转、裁剪和颜色变化，让模型在训练中接触更多情况，学习哪些变化应当保留同样的判断。

图像变换与标签

设 T 为随机图像变换，T_y 为对应的标签变换，增强后的训练目标可写为

$$\mathcal{L}_{\mathrm{aug}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \mathbb{E}_T \left[\ell\big(f_\theta(T(x_i)), T_y(y_i)\big) \right]. \tag{3.26}$$

对整图分类，水平翻转、适度裁剪和颜色扰动常常可以保留类别，此时 $T_y(y) = y$。是否保留标签，要看变换后的图像内容。例如，裁剪后应仍能看到目标；处理含文字或方向含义的图像时，翻转可能需要同步修改标签。

对于带空间标注的任务，图像的几何变换必须同时作用于标注。例如翻转图像时要变换检测框坐标，裁剪或缩放时要同步处理分割掩码。对离散类别掩码，插值方法也必须避免把类别编号混成没有含义的中间值。

训练阶段通常随机采样变换，评估阶段则固定处理流程。多裁剪或多尺度测试会对同一图像运行多次并平均预测，因此需要连同额外计算量一起报告。

Mixup

Mixup [21] 从两个训练样本 (x_i, q_i)、(x_j, q_j) 出发，其中 q 为 one-hot 类别向量，构造

$$\widetilde{x} = \lambda x_i + (1-\lambda)x_j, \qquad \widetilde{q} = \lambda q_i + (1-\lambda)q_j, \qquad \lambda \sim \mathrm{Beta}(\alpha, \alpha),\ \alpha > 0. \tag{3.27}$$

模型看到混合后的图像，同时接受按相同比例混合的标签监督。这会鼓励预测在两个训练样本之间平滑变化。Mixup 用人工构造的样本来施加这种训练要求。

交叉熵对目标分布是线性的，所以对同一组混合输入 logits，有

$$\ell_{\mathrm{CE}}(a, \widetilde{q}) = \lambda \ell_{\mathrm{CE}}(a, q_i) + (1-\lambda)\ell_{\mathrm{CE}}(a, q_j). \tag{3.28}$$

利用这个等式，可以直接计算两项类别损失，再按 λ 加权。α 控制混合比例：小于 1 时更常得到接近原图的样本，大于 1 时两张图混合得更均匀。可以通过验证集选择适合当前任务的混合强度。

算法实现 | Mixup 的实现

```
import torch.nn.functional as F

# x: [B, C, H, W]; y: [B], integer class indices
# model is in training mode; alpha is a positive scalar
beta = torch.distributions.Beta(alpha, alpha)
lam = beta.sample().item()
perm = torch.randperm(x.size(0), device=x.device)
mixed_x = lam * x + (1 - lam) * x[perm]
logits = model(mixed_x)
loss = (lam * F.cross_entropy(logits, y)
        + (1 - lam) * F.cross_entropy(logits, y[perm]))
```

这个 batch 共用一个 λ，图像和标签位于同一设备。模型对混合图像做一次前向计算，再分别计算两组标签的损失。若每个样本使用不同的混合系数，就先保留每个样本的损失，分别加权后再求平均。

CutMix 用另一张图像的局部区域替换原图，并按实际替换面积调整标签权重。随机擦除模拟局部遮挡，自动增强策略则选择变换的种类和强度。选择方法时，可以结合模型容易出错的情况，例如位置变化、颜色变化或局部遮挡，再检查增强后的标签是否仍然合适。

图文任务中的视觉特征

图文任务对视觉特征的要求各不相同。全局平均池化得到的 $z \in \mathbb{R}^C$ 适合表示整幅图像；区域与词语对齐时，通常保留 $U \in \mathbb{R}^{C \times H' \times W'}$，再把各空间位置展开为视觉 token。一个 token 对应一片感受野，相邻 token 的感受野可以重叠。

例如，图像的描述是“左侧有一只红色杯子”。水平翻转后，杯子仍属于同一类别，文字中的“左侧”却需要改为“右侧”；裁剪时还要保留杯子所在区域。图文训练因此需要一起处理图像和描述，让位置、颜色等细节继续对应。

3.8 目标检测与评价指标

前面介绍了怎样提取图像特征，并用这些特征判断整幅图像的类别。目标检测进一步回答物体的位置和数量，例如图中有几个人、分别在哪里。图像分割则把预测细化到像素，描出物体或区域的形状。



图 3.5 同一场景的四种输出形式。分类判断图中有哪些类别；检测给出各个目标的边界框；语义分割区分类别区域；实例分割进一步区分同类的不同个体。

检测输出

设图像中有 $n(X)$ 个目标，检测标注可以写为

$$\mathcal{G}(X) = \{(b_j^*, y_j^*)\}_{j=1}^{n(X)}, \qquad y_j^* \in \{1, \dots, K\}. \tag{3.29}$$

b_j^* 表示真实边界框，y_j^* 表示类别。模型也要为每个检测到的目标输出一个框、一个类别和相应分数。一张图像可能只有一个目标，也可能有很多目标，因此输出的数量需要随图像内容变化。标注中的目标可以任意编号，训练时还需把预测与真实目标对应起来。

常见检测器先设置许多候选位置或候选区域，再为每个候选预测类别和框的位置。训练时，将候选与真实框匹配，为它们分配监督。推理时，筛去背景和重复结果，剩下的框就组成检测集合。这样，网络内部可以使用固定形状的张量，最终输出的目标数则由筛选结果决定。

边界框常采用角点坐标 $b = (x_1, y_1, x_2, y_2)$，或中心与宽高 $b = (x_c, y_c, w, h)$。两者可以互相转换，例如 $x_c = (x_1 + x_2)/2$、$w = x_2 - x_1$。以下采用连续坐标，面积按宽乘高计算，并要求 $x_2 > x_1, y_2 > y_1$。图像缩放、裁剪或填充后，框的坐标也随之变换。

交并比（IoU）

正确的检测需要同时给出物体类别和位置。交并比（intersection over union，IoU）用预测框与真实框的重叠程度衡量定位效果，定义为

$$\mathrm{IoU}(b, b^*) = \frac{|b \cap b^*|}{|b \cup b^*|} = \frac{I}{|b| + |b^*| - I}. \tag{3.30}$$

对角点形式，交集宽度为 $\max(0, \min(x_2, x_2^*) - \max(x_1, x_1^*))$，高度同理，二者相乘得到 I。完全重合时 IoU 为 1，互不相交时为 0。它以区域比例度量误差，能够区分同样的坐标偏移对大框和小框造成的不同影响。

例如，两个面积均为 4 的正方形重叠面积为 1，则 IoU 为 $1/(4+4-1) = 1/7$。评估时，会先检查类别是否一致，再检查 IoU 是否达到规定阈值。

非极大值抑制（NMS）

相邻候选经常指向同一个物体。若每个候选都单独保留，模型可能围绕一只猫输出十几个框。非极大值抑制（non-maximum suppression，NMS）用分数与框间重合程度处理这个问题：先保留当前分数最高的框，再抑制与它的 IoU 超过阈值 τ_{NMS} 的其他框，然后在剩余集合中重复。

常见实现按类别分别执行 NMS。较低的阈值会删去更多重叠框，较高的阈值则保留更多候选。在拥挤场景中，两个同类目标也可能高度重叠，较强的抑制就容易误删其中一个。选择阈值时，需要结合重复框和漏检的情况。

> **备注 | 检测中的阈值**
> 训练匹配阈值比较候选框与真实框，用于分配监督。NMS 阈值比较预测框之间的重叠，用于去重。评价阈值再将最终预测与真实框比较，用于计分。它们各自设定；此外，还可以用置信分数阈值筛去低分预测。

AP 与 mAP

在一个给定类别和 IoU 阈值下，先按分数对检测结果排序，再将每个预测与同一图像中尚未匹配的真实目标对应。忽略特殊的 crowd 或 ignore 标注时，一个真实目标最多贡献一次真阳性（TP）；围绕已经匹配的目标重复预测，仍会形成假阳性（FP）。没有被检测到的真实目标计为假阴性（FN）。于是

$$\mathrm{Precision}=\frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}},\qquad \mathrm{Recall}=\frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}}. \tag{3.31}$$

按分数从高到低逐步纳入预测，就能得到一系列精确率和召回率。AP 汇总这条曲线，可以用 $\int_0^1 P_{\mathrm{interp}}(r)\,\mathrm{d}r$ 理解，实际评价通常采用规定的插值与离散采样方法。高分预测越准确，模型越容易在较高召回率下保持较高精确率，AP 也就越高。

对各类别的 AP 取平均得到 mAP。PASCAL VOC 常用 IoU 0.5 判断匹配，具体插值规则随评价版本而定。COCO 的主要检测 AP 还要对 $0.50, 0.55, \ldots, 0.95$ 十个 IoU 阈值取平均，因此它同时考察多个定位精度要求。[37] 报告结果时应注明采用 AP_{50} 还是 COCO AP。

常用 VOC 检测任务有 20 类，COCO 检测任务有 80 类。它们的目标尺度、遮挡和场景分布也有所不同，比较模型时需要使用同一数据集及评价设置。

速度指标同样需要明确条件。FPS 随硬件、输入分辨率、批量大小和是否计入预处理及 NMS 而变化。把检测精度与速度放在一起比较，才能判断模型在给定任务和计算预算下是否合适。

3.9 R-CNN 系列

对大量图像窗口逐一分类是一种直接的检测思路,但重叠窗口会反复计算同一片图像。R-CNN 系列围绕这项开销逐步改进：先减少需要观察的区域，再共享区域之间的卷积特征，最后把候选区域的生成也交给网络。

R-CNN

区域提议方法先寻找可能包含物体的区域，而不必立即确定其类别。Selective Search 从图像的初始区域出发，依据颜色、纹理、大小和空间关系等相似性逐步合并，产生不同尺度的候选框。它希望以相对少的候选覆盖尽可能多的真实目标；如果某个目标没有得到合适的候选，后续分类器再强也很难补救。

R-CNN [22] 将各个候选图像区域裁出并缩放到 CNN 接受的尺寸，分别提取特征，再进行分类和边界框回归。原始流程包括使用分类目标微调 CNN、训练类别 SVM，以及训练框回归器等阶段。背景样本让分类器学会拒绝不包含目标的区域，框回归则修正候选与真实位置之间的偏差。

这种流程把分类预训练的视觉特征引入检测,但每幅图像中的大量候选都需要单独通过 CNN,重叠区域的特征被重复计算。分阶段训练还涉及特征缓存和不同目标函数，增加了训练和维护成本。

Fast R-CNN

Fast R-CNN [23] 先让整幅图像通过主干网络得到共享特征图，再把候选框映射到这张特征图。每个区域通过 RoI Pooling 转成固定形状的特征，随后同时预测类别和坐标修正。昂贵的主干卷积因此只需对整幅图像计算一次。

设共享特征为 $F \in \mathbb{R}^{C\times H_f\times W_f}$，每幅图像共有 R 个候选，区域算子将不同大小的候选统一为 $[R, C, h_r, w_r]$。原始 Fast R-CNN 的类别头输出 $K+1$ 类分布，其中额外一类表示背景；回归头则学习目标框的修正。候选区域仍来自外部算法，因此共享卷积之后，区域提议本身又成为明显的开销。

RoI Pooling 与 RoI Align

RoI Pooling 把一个区域划分为固定数量的网格，并在每格内进行最大池化。区域大小即使不同，最终输出的网格数也相同。但候选坐标从原图映射到特征图时可能是小数，区域边界和网格边界的离散化会改变其实际覆盖位置。

假设特征图相对图像的采样间隔为 32，原图坐标 665 映射为 $665/32 = 20.78125$。若向下取整到 20，再换算回原图，就是坐标 640，偏移了 25 像素。这种坐标误差对小目标尤其明显。

RoI Align 在连续坐标上保留区域与采样点的位置，使用双线性插值读取特征。对落在四个整数网格点之间的位置 (x, y)，令 $x_0 = \lfloor x\rfloor$、$y_0 = \lfloor y\rfloor$、$a = x - x_0$、$b = y - y_0$，则单个通道的插值为

$$\begin{aligned} F(x,y) = {} & (1-a)(1-b)F(x_0,y_0) + a(1-b)F(x_0+1,y_0) \\ & + (1-a)bF(x_0,y_0+1) + abF(x_0+1,y_0+1). \end{aligned} \tag{3.32}$$

每个区域网格可以取多个采样点，再将插值结果平均或取最大值。这样，区域边界和采样位置可以保留小数坐标，抽取的特征也与原区域更好地对齐。

RoI Align 由 Mask R-CNN [25] 提出，主要改善像素对齐，对掩码预测很有帮助。原始 Fast R-CNN 和 Faster R-CNN 使用 RoI Pooling，后续实现也常采用 RoI Align。

Faster R-CNN

Faster R-CNN [24] 在共享特征图上加入区域提议网络（region proposal network，RPN）。每个空间位置先放置 A 个不同尺度或长宽比的参考框，称为锚框（anchor）。RPN 为每个锚框预测目标性分数，判断它是否包含物体，再预测四个修正量来调整位置和大小。

若特征图为 $H_f \times W_f$，候选参考框总数为 AH_fW_f。使用单个二分类 logit 时，目标性输出为 $[A, H_f, W_f]$，回归输出为 $[4A, H_f, W_f]$；若采用前景、背景两个 logits，分类输出就是 $[2A, H_f, W_f]$。例如 $H_f = 20, W_f = 15, A = 9$ 时，RPN 对 2 700 个参考框做稠密预测。

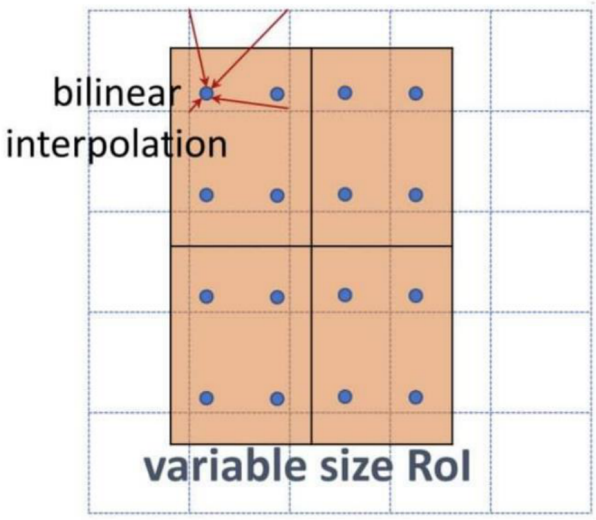


图 3.6 RoI Align 在每个网格内设置连续采样点，再由邻近特征插值得到响应。图中的蓝点是采样位置，虚线表示底层特征网格。

算法实现 | RoI Align 的实现

```python
from torchvision.ops import roi_align

# feat: [B, C, Hf, Wf], computed at image stride 16
# rois: [R, 5] = [batch_index, x1, y1, x2, y2]
# coordinates refer to the resized/padded network input
pooled = roi_align(feat, rois, output_size=(7, 7),
                   spatial_scale=1 / 16, sampling_ratio=2,
                   aligned=True)   # [R, C, 7, 7]
```

这里每格取 2×2 个采样点并平均。`aligned=True` 采用半像素偏移，需要与模型训练时的坐标约定一致。框采用图像坐标时，用 `spatial_scale` 映射到特征图；框已经换算为特征图坐标时，取 `spatial_scale=1`。[36]

训练时，根据锚框与真实框的重叠程度，将锚框分为正样本、负样本和忽略样本。正样本接受目标性和坐标监督，负样本接受背景监督，忽略样本跳过相应损失。匹配阈值与正负样本比例由训练设置确定。

推理时，RPN 根据修正量生成框，裁剪越界部分，再通过分数筛选和 NMS 保留候选。第二阶段从共享特征图中提取这些区域的特征，预测具体类别并进一步修正框。两阶段共用一次主干计算，区域头则可以把多个候选放在同一批张量中处理。

训练目标通常包含四类监督，可概括为

$$\mathcal{L} = \mathcal{L}_{\mathrm{RPN,cls}} + \lambda_1 \mathcal{L}_{\mathrm{RPN,box}} + \mathcal{L}_{\mathrm{RoI,cls}} + \lambda_2 \mathcal{L}_{\mathrm{RoI,box}}. \tag{3.33}$$

各项损失分别按对应样本数或其他约定归一化。共享主干接收来自 RPN 和区域预测头的梯度，两者一起训练。候选筛选、排序和 NMS 则按离散规则执行。

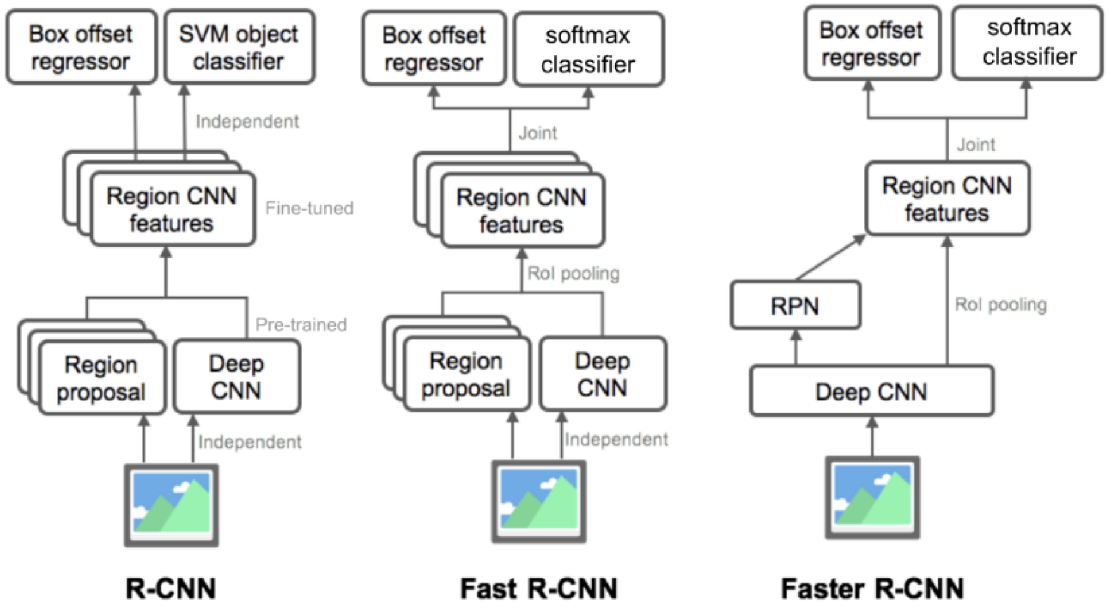


图 3.7 R-CNN 系列如何逐步共享计算。R-CNN 对候选区域分别提取 CNN 特征；Fast R-CNN 先计算整图特征；Faster R-CNN 进一步把候选生成换成与检测头共享主干的 RPN。

3.10 YOLO 系列

两阶段检测先缩小需要精细判断的区域范围，再分配较多计算处理候选。另一种选择是在特征图上直接预测最终的类别和边界框，省去逐候选的区域特征抽取，这形成了单阶段检测思路。它的困难随之转移到稠密预测的组织方式：一个位置负责哪个目标、大量背景怎样参与训练，以及不同尺度的物体怎样得到足够的特征支持。

YOLO v1 的输出

YOLO v1 [26] 将图像划分为 $S \times S$ 个网格，由目标中心落入的网格负责预测该目标。每个网格输出 Q 组边界框，每组包含四个坐标量与一个置信度，同时共享一组 K 维的条件类别分数。因此输出形状为

$$S \times S \times (5Q + K). \tag{3.34}$$

在 VOC 的原始设置中，$S = 7, Q = 2, K = 20$，输出为 $7 \times 7 \times 30$，共包含 98 个候选框。其中 Q 是每格的框数，B 仍表示批量大小。20 个类别对应物体类别，空网格通过低置信度表示背景。

每个框的中心坐标相对于所在网格表示，宽高则相对于整幅图像归一化。训练时，每个有目标的网格选出与真实框当前 IoU 最高的预测器负责定位。尽管一格输出多个候选框，原始的监督分配与共享类别设计仍限制了同格中多个目标的表达，尤其不利于中心接近的小目标。

YOLO v1 直接从网格特征预测框。后来的锚框版本会先给出参考宽高，再学习相对于参考框的调整；下面介绍 YOLOv2 时会用到这种写法。

置信度与训练目标

YOLO v1 的框置信度 o 结合目标存在与定位质量，写为 $\Pr(\text{object})\,\mathrm{IoU}(b, b^{*})$。令 p_k 为网格预测的条件类别分数，推理时使用

$$s_k = p_k\, o \tag{3.35}$$

作为该框的类别得分，再进行阈值筛选和 NMS。推理时，p_k 和 o 都由网络输出，乘积用于排序和筛选。这个分数同时包含类别和定位质量的信息。

原始 YOLO v1 使用平方误差组合定位、置信度和类别监督，可按作用写为

$$\mathcal{L} = \lambda_{\text{coord}}\mathcal{L}_{\text{loc}} + \mathcal{L}_{\text{conf,pos}} + \lambda_{\text{noobj}}\mathcal{L}_{\text{conf,neg}} + \mathcal{L}_{\text{class}}. \tag{3.36}$$

定位损失只施加给负责目标的预测器，类别损失只施加于有目标的网格，背景置信度目标为零。原始设置取 $\lambda_{\text{coord}} = 5$、$\lambda_{\text{noobj}} = 0.5$，提高定位项的权重并降低大量背景项的影响。

其中宽高使用平方根后的误差，例如 $(\sqrt{\hat{w}} - \sqrt{w^*})^2$。在 $\hat{w}$ 接近 $w^* > 0$ 时，泰勒展开给出

$$(\sqrt{\hat{w}} - \sqrt{w^*})^2 \approx \frac{(\hat{w} - w^*)^2}{4w^*}.$$

分母中的 w^* 越小，同样的宽度误差受到的惩罚越大。因此，平方根写法提高了小框宽高误差在训练中的相对权重。这是 YOLO v1 的设计，后续版本采用了其他回归目标。

YOLOv2

YOLOv2 [27] 使用 Darknet-19 主干、批量归一化及全卷积检测头，并引入锚框。类别预测也从“每个网格一组”变为“每个锚框一组”。若每位置采用 A 个锚框，一个常见输出布局为 $[A(5+K), H_f, W_f]$，与式 (3.34) 的结构不同。

锚框形状可以从训练集的框宽高中聚类得到。使用 $1 - \text{IoU}$ 作为形状距离，有助于避免单纯欧氏距离过度受大框尺寸支配。给定网格索引 (i_x, i_y) 和以特征网格为单位的锚框宽高 (a_w, a_h)，YOLOv2 的核心解码形式为

$$\hat{x}_c = i_x + \text{sigmoid}(t_x), \quad \hat{y}_c = i_y + \text{sigmoid}(t_y), \quad \hat{w} = a_w e^{t_w}, \quad \hat{h} = a_h e^{t_h}. \tag{3.37}$$

sigmoid 将中心限制在对应网格内，指数变换保证宽高为正。上式采用特征网格的单位；乘以特征步长后得到输入图像上的像素坐标。若以整幅图像的宽高为单位进行归一化，则横向量除以 W_f，纵向量除以 H_f。

较深层的特征适合识别物体类别，较浅层则保留更多位置细节。YOLOv2 的 passthrough 层把较浅层特征中相邻的位置排列到通道维，例如将 $[512, 26, 26]$ 变为 $[2048, 13, 13]$，再与低分辨率特征拼接。重排保持元素总数不变，将这些细节一并交给检测头。

原论文常用 416×416 检测输入，对应步长 32 下的 13×13 网格，并在训练中使用多种输入尺寸。提高分辨率可以给小目标提供更多像素，同时也增加激活存储和计算量。

YOLO9000、YOLOv3 与 YOLOv4

YOLO9000 与 YOLOv2 来自同一项工作，它利用分类数据扩展可识别的类别。WordTree 将类别按层次组织，例如把较具体的类别放在其父类下面。模型预测子类在父类条件下的概率，再沿路径相乘得到完整概率。检测样本提供类别和位置监督，只有分类标签的样本则参与类别训练。

YOLOv3 [28] 采用带残差结构的 Darknet-53，在多个分辨率上同时预测目标。高分辨率特征有利于定位较小的物体，低分辨率特征则覆盖较大的图像范围，网络将这些尺度的特征结合起来。

多尺度预测使用同一次前向中的不同特征图，多尺度训练则改变输入图像的尺寸。两者可以同时使用。YOLOv3 的类别头为各类别分别预测 logistic 分数。

YOLOv4 [29] 则强调主干、特征融合、预测头和训练策略的整体配合。理解其中的训练方法，可以联系前面的数据增强与正则化：Mosaic 把多幅图像拼接成一个训练样本，所有框随几何变换

同步调整；DropBlock 在特征图中遮去连续区域，减少相邻位置相互替代而削弱随机失活效果的问题；标签平滑把过于尖锐的监督分布适当分散。所谓 bag of freebies，指主要在训练阶段使用、推理时通常不增加相应分支的改进。

自对抗训练分为两步。先固定模型参数，根据当前模型的反应修改图像，使目标更难被检测。再用修改后的图像训练检测器，让它学习应对这种扰动。

这些版本分别改进了框的预测方式、不同尺度特征的使用，以及训练样本的构造。阅读具体模型时，可以沿这几项设计查看它的计算过程。

3.11 检测损失函数

检测器要同时学会判断类别和调整框的位置。由于大多数候选属于背景，训练还需要处理正负样本数量的差异。本节先介绍框回归，再讨论分类损失如何关注困难样本，最后介绍直接使用框的几何关系计算的损失。

边界框回归

对参考框 $b_a = (x_a, y_a, w_a, h_a)$ 与匹配的真实框 $b^* = (x^*, y^*, w^*, h^*)$，R-CNN 系列常用的回归目标为

$$t_x^* = \frac{x^* - x_a}{w_a}, \quad t_y^* = \frac{y^* - y_a}{h_a}, \quad t_w^* = \log \frac{w^*}{w_a}, \quad t_h^* = \log \frac{h^*}{h_a}. \tag{3.38}$$

中心位移除以参考宽高，尺寸变化写成比例的对数，使大框和小框使用相近的回归尺度。网络预测 $\hat{t}$ 后，再将它换回框坐标：$\hat{x} = x_a + w_a \hat{t}_x$、$\hat{y} = y_a + h_a \hat{t}_y$、$\hat{w} = w_a e^{\hat{t}_w}$、$\hat{h} = h_a e^{\hat{t}_h}$。这套写法允许中心自由偏移；YOLOv2 的式 (3.37) 则通过 sigmoid 将中心限制在网格内。

Fast R-CNN 使用 Smooth L1 对编码后的误差进行惩罚。取转折点为 1 时，单个分量的损失为

$$\rho(u) = \begin{cases} \frac{1}{2} u^2, & |u| < 1, \\ |u| - \frac{1}{2}, & |u| \geq 1. \end{cases} \tag{3.39}$$

误差较小时使用平方项，接近目标后可以平滑地调整；误差较大时使用线性项，使惩罚增长得较缓。对四个坐标的误差分别计算并求和，得到框回归损失。它只施加于匹配到真实目标的正样本，背景样本接受分类监督。

Focal Loss

许多背景候选很快就能被正确分类。它们各自的损失虽小，数量却很多，加起来可能超过少量困难样本的损失。Focal Loss [30] 为容易样本减小权重，使训练更多地关注尚未分对的样本。对二分类标签 $y \in \{0, 1\}$ 和预测概率 p，定义

$$p_t = \begin{cases} p, & y = 1, \\ 1 - p, & y = 0, \end{cases} \qquad \mathcal{L}_{\text{focal}} = -\alpha_t (1 - p_t)^{\gamma} \log p_t, \tag{3.40}$$

其中 $\gamma \geq 0$，α_t 用于平衡类别。取 $\alpha_t = \alpha$ 对应正类、$1 - \alpha$ 对应负类时，α 与 γ 的作用不同：前者调节类别权重，后者调节容易与困难样本的相对贡献。

当 $\gamma = 2$ 时，$p_t = 0.99$ 的样本乘以 10^{-4}，$p_t = 0.5$ 的样本乘以 0.25，容易样本的权重下降得更多。取 $\gamma = 0$ 时，调制因子等于 1，得到带 α_t 权重的交叉熵。反向传播对整个表达式求导，包括随预测变化的调制因子。

Focal Loss 最初用于 RetinaNet，通常对各类别分别计算二分类损失。训练仍需先完成候选与目标的匹配。较大的 γ 会更集中地关注困难样本，其中也可能包含错标，因此可以结合数据质量和验证结果选择它。

算法实现 | Focal Loss 的实现

```
import torch
import torch.nn.functional as F

def focal_binary(logits, targets, alpha=0.25, gamma=2.0):
    ce = F.binary_cross_entropy_with_logits(
        logits, targets, reduction="none")
    p = torch.sigmoid(logits)
    pt = targets * p + (1 - targets) * (1 - p)
    at = targets * alpha + (1 - targets) * (1 - alpha)
    return at * (1 - pt).pow(gamma) * ce
```

目标使用与 logits 同形状的浮点二元张量，函数返回每个元素的损失。检测训练先排除忽略候选，再对有效项求和，并按约定的样本数归一化。代码使用带 logits 的 BCE 函数来稳定计算对数部分。

IoU Loss 与 GIoU

对编码坐标做回归能够稳定地学习修正量，但相同坐标误差可能对应不同的区域重合。IoU Loss 直接使用

$$\mathcal{L}_{\mathrm{IoU}} = 1 - \mathrm{IoU}(b, b^*). \tag{3.41}$$

这一损失与定位评价更接近，但在两框严格分离且小幅移动仍不相交的区域内，IoU 恒为零，单靠此项无法提供把它们拉近的梯度。几何改进的目的，就是在重合之外加入其他能反映偏差的信息。

GIoU [31] 使用同时包围两框的最小轴对齐外接矩形 E，定义

$$\mathcal{L}_{\mathrm{GIoU}} = 1 - \mathrm{IoU}(b, b^*) + \frac{|E| - |b \cup b^*|}{|E|}. \tag{3.42}$$

新增项惩罚外接矩形中没有被两框覆盖的空白区域，使不相交的框也可能得到有效的几何信号。不过，若一个框完全包含另一个框，外接矩形就等于并集，新增项为零；在内框面积不变且始终被包含时，移动内框也不会改变 IoU。这是仅依赖区域面积仍可能遇到的局限。

DIoU 与 CIoU

DIoU [32] 在 IoU 项之外直接惩罚中心距离。设两个框的中心为 c, c^*，最小外接矩形的对角线长度为 d_E，则

$$\mathcal{L}_{\mathrm{DIoU}} = 1 - \mathrm{IoU}(b, b^*) + \frac{\|c - c^*\|_2^2}{d_E^2}. \tag{3.43}$$

对角线归一化使距离项具有相对尺度意义。即使内框在外框中移动时 IoU 不变，中心距离仍能区分它离正确位置有多远。

CIoU 进一步加入长宽比项：

$$v = \frac{4}{\pi^2}\left(\arctan\frac{w^*}{h^*} - \arctan\frac{w}{h}\right)^2, \qquad \alpha = \frac{v}{1 - \mathrm{IoU}(b, b^*) + v}, \tag{3.44}$$

$$\mathcal{L}_{\mathrm{CIoU}} = \mathcal{L}_{\mathrm{DIoU}} + \alpha v. \tag{3.45}$$

这里的 α 是几何损失内部的系数，与 Focal Loss 的类别权重无关。框必须具有正宽高；完全重合时按连续延拓取 $\alpha v = 0$，实现中通常还为分母加入数值保护。

长宽比项调整框的形状比例。例如，两个框中心相同、比例也相同，只有大小不同，此时中心距离和 v 都为零，大小由 IoU 项继续调整。GIoU 使用外接区域，DIoU 使用中心距离，CIoU 再加入比例信息；它们提供了不同的几何误差信号。

3.12 语义分割与 FCN

边界框给出物体的大致范围，语义分割进一步给每个像素赋予类别，描出物体轮廓。如果图中有两只猫，语义分割将它们的像素都标为“猫”。实例分割则还会把这些像素分成两组，分别对应两只猫。

全卷积网络

最直接的分割方法是以每个像素为中心裁出一个窗口，用分类器预测中心像素的类别。但相邻窗口大量重叠，重复卷积会带来与早期检测方法相似的开销。全卷积网络（fully convolutional network，FCN）[33] 在整幅图像上共享计算，直接输出空间上的类别分数图。

设输入为 $[B, C, H, W]$，最终 logits 为 $Z \in \mathbb{R}^{B\times K\times H\times W}$，其中 K 表示所有待预测的像素类别，必要时包含背景。网络末端可用 1×1 卷积把每个位置的特征映射为 K 个分数。推理时沿类别维取最大值，得到

$$\hat{y}_{b,i,j} = \arg\max_{k\in\{0,\dots,K-1\}} Z_{b,k,i,j}. \tag{3.46}$$

这里采用从 0 开始的类别索引，与后面的代码一致。训练时将连续 logits 交给损失函数，推理时再通过 argmax 生成类别图。

全卷积网络在整幅图像上重复使用同一组卷积参数，保留各位置的特征。为减少计算量，网络通常先下采样，在较小的特征图上提取语义，再上采样到目标分辨率。较浅层保留的高分辨率特征，也可以与深层特征融合，帮助恢复边界。

像素损失

对有效像素集合 Ω，多类交叉熵为

$$\mathcal{L}_{\mathrm{seg}} = -\frac{1}{|\Omega|}\sum_{(b,i,j)\in\Omega} \log\frac{\exp Z_{b,y_{b,i,j},i,j}}{\sum_{k=0}^{K-1}\exp Z_{b,k,i,j}}. \tag{3.47}$$

Ω 只包含有监督的像素。未标注区域、填充区域和指定的忽略类别从损失中排除。标签保存为 $[B, H, W]$ 的整数类别索引，logits 则为每个位置保存 K 个分数，因此多出一个类别维度。

背景或大面积区域常常占据多数像素，模型容易偏向这些类别。可以提高少数类的损失权重，或使用 Focal Loss 关注困难像素。还可以直接比较预测区域与标注区域的重合程度，下面的 Dice 损失就是一种做法。

算法实现 | 分割损失的实现

```
import torch.nn.functional as F

# logits: [B, K, H, W]; target: [B, H, W], int64
# valid labels: 0,...,K-1; 255 denotes ignored pixels
valid = target.ne(255)
pixel_loss = F.cross_entropy(
    logits, target, ignore_index=255, reduction="none")
loss = (pixel_loss * valid).sum() / valid.sum().clamp_min(1)
pred = logits.argmax(dim=1)          # [B, H, W]
```

这里按整个 batch 的有效像素平均。若所有像素都被忽略，应跳过该批次的参数更新。几何增强时同步变换图像和标签；缩放离散类别图通常使用最近邻插值，避免产生混合后的非法类别编号。

例如，对某个类别的预测概率 p_i 和二元标注 g_i，一种常见的 soft Dice 损失为

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2\sum_i p_i g_i + \epsilon}{\sum_i p_i + \sum_i g_i + \epsilon}. \tag{3.48}$$

式中对有效像素求和，$\epsilon > 0$ 用于稳定分母。Dice 将整片预测区域与标注区域比较，因此一个像素的影响也与其他像素有关。实现时需要说明按单图还是按整个 batch 计算，以及背景和空类别如何处理。

mIoU

评价时，对每个类别 k 把像素集合视为区域，计算

$$\text{IoU}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k + \text{FN}_k}, \qquad \text{mIoU} = \frac{1}{|\mathcal{K}_{\text{eval}}|} \sum_{k \in \mathcal{K}_{\text{eval}}} \text{IoU}_k \,. \tag{3.49}$$

这里 TP、FP、FN 统计像素数量，$\mathcal{K}_{\text{eval}}$ 是参与评价的类别集合。通常先在数据集上累计每类的像素计数，再计算各类 IoU。背景是否参与平均、空类别怎样处理，应按所用数据集的评价方法确定。

mIoU 评价类别区域的重合。实例分割还要判断同类物体是否被分成了独立目标，因此通常为每个目标单独预测掩码。例如，Mask R-CNN 在检测区域的特征上增加掩码分支，利用 RoI Align 保持区域与像素的对应。

3.13 上采样与 U-Net

下采样让网络用较少的位置汇集图像内容，分割时则需要重新生成较大的类别图。上采样首先增加输出位置，再通过卷积和高分辨率特征补充这些位置的预测信息。

插值与反池化

最近邻上采样复制相邻位置的值，双线性上采样对邻近四点加权，计算中间位置的值。两者都使用固定规则，常在后面接可学习的卷积。恢复边缘等细节时，还可以结合编码器保存的高分辨率特征。

最大反池化（max unpooling）则利用最大池化时保存的位置索引，把响应放回较大网格中的相应位置，其余位置通常置零。若一个池化窗口为

$$\begin{pmatrix} 1 & 6 \\ 3 & 2 \end{pmatrix},$$

最大池化保存数值 6 及其位置。反池化把响应放回这个位置，得到 $\left(\begin{smallmatrix} 0 & 6 \\ 0 & 0 \end{smallmatrix}\right)$，其余三个位置补零。这一过程保留了最大值的位置，原来的其他数值已在池化时丢失。若中间网络改变了响应，反池化就按保存的位置放入更新后的值。

转置卷积

忽略偏置和非线性，卷积可以写成稀疏矩阵乘法 $y = Ax$。转置卷积采用 $A^\top$，沿相反的连接方向将输入响应分配到输出位置，重叠处相加。以一维长度为 3 的核 (a, b, c)、步长 2、无填充为例：

$$A = \begin{pmatrix} a & b & c & 0 & 0 \\ 0 & 0 & a & b & c \end{pmatrix}, \qquad A^\top \begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} au \\ bu \\ cu + av \\ bv \\ cv \end{pmatrix}. \tag{3.50}$$

每个输入响应缩放一份核，核在输出上的相对间隔由步长决定，重叠处求和。这个形式也解释了为什么卷积对输入的反向传播涉及转置卷积：$\partial \mathcal{L}/\partial x = A^\top (\partial \mathcal{L}/\partial y)$。

矩阵转置描述连接方向的变化。正向卷积因下采样等操作丢失的信息，需要解码网络从保留的特征中推断。在编码器和解码器中，两边的卷积参数通常各自学习。

对单个空间方向，转置卷积输出长度为

$$L_{\text{out}} = (L_{\text{in}} - 1)s - 2p + \delta(k - 1) + o + 1, \tag{3.51}$$

其中 s, p, δ, k 分别表示步长、填充参数、膨胀率和核大小，o 对应 `output_padding`。例如 $L_{\text{in}} = 2, k = 3, s = 2, p = 1, \delta = 1$ 时，取 $o = 0$ 得到长度 3，取 $o = 1$ 得到长度 4。输出尺寸由这些设置共同确定。

步长卷积可能将不同长度的输入变成同样长度的输出，`output_padding` 用来选择转置卷积对应的输出尺寸。[36] 编码器多次下采样后若出现奇数尺寸，可以逐层计算，再用裁剪、填充或指定目标大小的方式对齐特征。

转置卷积的不同输出位置还可能接收不同数量的重叠贡献，形成周期性的强弱差异，即常说的棋盘格伪影。先插值到目标大小再卷积，是另一种常见选择。哪一种上采样更合适，需要结合输出任务、对齐约定和实际训练效果判断。

U-Net

如果只从最深层的低分辨率特征恢复分割图，网络需要重新推断许多已经被压缩的细节。U-Net [34] 因此采用编码器与解码器相配合的结构：编码器逐级下采样、提取上下文，解码器逐级上采样，并在每个尺度接入编码器保留的特征。

设编码器在第 ℓ 个尺度的特征为 E_ℓ，较粗尺度的解码特征为 $D_{\ell+1}$，则典型解码步骤可写为

$$D_\ell = \mathcal{C}_\ell([\,E_\ell\,;\,\mathrm{Up}(D_{\ell+1})\,]_{\text{channel}})\,. \tag{3.52}$$

上采样恢复空间尺寸，拼接把较浅层的定位信息与较深层的语义信息并列提供，后续卷积再学习如何融合。若两支分别为 $[B, C_e, H, W]$ 与 $[B, C_d, H, W]$，拼接后为 $[B, C_e + C_d, H, W]$。空间尺寸必须一致，通道数则可以不同。

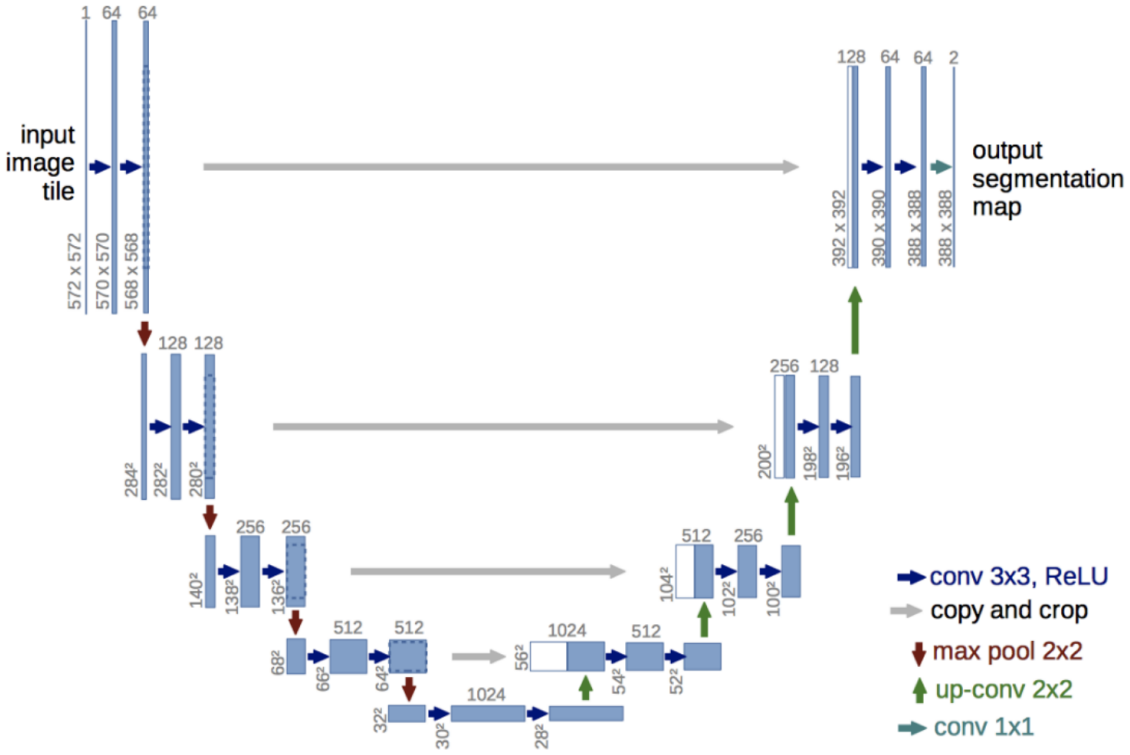


图 3.8 原始 U-Net 的编码与解码路径 [34]。灰色横向路径将编码特征裁剪并拼接到解码端；原图中的通道数与空间尺寸展示了一套具体配置。

U-Net 的横向连接将编码与解码特征沿通道拼接。两边只需空间尺寸一致，拼接后再由卷积混合通道。ResNet 的捷径则逐元素相加，需要两边的通道数也一致。实际模型还可以在 U-Net 的卷积块内部使用残差连接。

原始 U-Net 使用无填充卷积，每次卷积都会缩小特征图，因此横向拼接前先裁剪编码特征。经典结构的 572×572 输入对应 388×388 输出。后续实现也常用保持尺寸的填充卷积，使输入与输出具有相同高宽。

3D U-Net 将卷积扩展到体数据，Attention U-Net 在跳跃分支中加入门控，UNet++ 则使用更密集的跨尺度连接。[35] nnU-Net 根据数据配置预处理、网络和训练流程。TransUNet、Swin-UNETR 等模型在特征提取中加入注意力，利用较远区域的信息。

检测、分割与多模态学习

检测框和分割掩码可以为图文任务提供位置。例如，模型可以先找到“杯子”的区域，再用该区域的特征回答颜色问题；需要精确轮廓时，则可以使用分割掩码。区域特征也可以作为视觉 token，交给后续语言模型。

“左边骑马的人”还包含相对位置和人与马之间的关系。模型先识别人和马，再结合“左边”“骑着”等词语选择符合描述的目标。这类任务把检测得到的空间信息与语言条件联系起来。

参考文献

[1] Baltrušaitis, Ahuja, Morency. Multimodal Machine Learning: A Survey and Taxonomy. 五类核心问题的分类框架。

[2] Hendrycks, Gimpel. Gaussian Error Linear Units (GELUs). GELU 的定义与门控解释。

[3] Shazeer. GLU Variants Improve Transformer. 门控前馈层与参数预算。

[4] Ioffe, Szegedy. Batch Normalization. BN 的统计量与可学习仿射变换。

[5] Ba, Kiros, Hinton. Layer Normalization. 逐样本归一化。

[6] Zhang, Sennrich. Root Mean Square Layer Normalization. RMSNorm。

[7] Peebles, Xie. Scalable Diffusion Models with Transformers. AdaLN 与 AdaLN-Zero。

[8] Loshchilov, Hutter. Decoupled Weight Decay Regularization. AdamW 与解耦权重衰减。

[9] PyTorch 官方文档：LayerNorm、BatchNorm2d、Dropout。

[10] PyTorch 官方文档：Conv2d。互相关、输出形状与分组卷积的实现约定。

[11] Simonyan, Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. 小卷积核与深层网络。

[12] Szegedy 等. Going Deeper with Convolutions. Inception 模块与 GoogLeNet。

[13] He, Zhang, Ren, Sun. Deep Residual Learning for Image Recognition. 残差学习与深层网络优化。

[14] Hu 等. Squeeze-and-Excitation Networks. 基于全局上下文的通道重标定。

[15] Howard 等. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. 深度可分离卷积。

[16] Tan, Le. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. 深度、宽度与分辨率的联合缩放。

[17] Zoph, Le. Neural Architecture Search with Reinforcement Learning. 基于控制器与验证奖励的架构搜索。

[18] Ding, Zhang, Han, Ding. Scaling Up Your Kernels to 31x31: Revisiting Large Kernel Design in CNNs. 大核逐通道卷积与结构重参数化。

[19] Chen, Chu, Ren, Zhao, Huang. PeLK: Parameter-efficient Large Kernel ConvNets with Peripheral Convolution. 大核内部的外围参数共享。

[20] Lou, Yu. OverLoCK: An Overview-first-Look-Closely-next ConvNet with Context-Mixing Dynamic Kernels. 上下文指导与逐位置动态卷积。

[21] Zhang, Cisse, Dauphin, Lopez-Paz. mixup: Beyond Empirical Risk Minimization. 输入与监督的联合插值。

[22] Girshick, Donahue, Darrell, Malik. Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation. R-CNN 与区域特征。

[23] Girshick. Fast R-CNN. 整图共享卷积、RoI Pooling 与多任务训练。

[24] Ren, He, Girshick, Sun. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. 可学习的区域提议。

[25] He, Gkioxari, Dollár, Girshick. Mask R-CNN. RoI Align 与实例掩码预测。

[26] Redmon, Divvala, Girshick, Farhadi. You Only Look Once: Unified, Real-Time Object Detection. YOLO v1 的网格输出与联合目标。

[27] Redmon, Farhadi. YOLO9000: Better, Faster, Stronger. YOLOv2、锚框、多尺度训练与 WordTree。

[28] Redmon, Farhadi. YOLOv3: An Incremental Improvement. 多尺度预测与 Darknet-53。

[29] Bochkovskiy, Wang, Liao. YOLOv4: Optimal Speed and Accuracy of Object Detection. 检测架构与训练策略。

[30] Lin, Goyal, Girshick, He, Dollár. Focal Loss for Dense Object Detection. 容易样本的损失调制。

[31] Rezatofighi 等. Generalized Intersection over Union: A Metric and A Loss for Bounding Box Regression. 不相交区域的几何监督。

[32] Zheng 等. Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression. DIoU 与 CIoU。

[33] Long, Shelhamer, Darrell. Fully Convolutional Networks for Semantic Segmentation. 全卷积密集预测与特征融合。

[34] Ronneberger, Fischer, Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation. 编码器、解码器与跨尺度拼接。

[35] Zhou 等. UNet++; Isensee 等. nnU-Net. 跳跃路径重组与分割流程的自动配置。

[36] PyTorch 与 Torchvision 官方文档：roi_align、sigmoid_focal_loss、ConvTranspose2d。

[37] COCO 官方检测评价实现。IoU 阈值、召回率采样、预测匹配与 AP 汇总规则。